

AIAA 2205: Introduction to AI

Enyan Dai

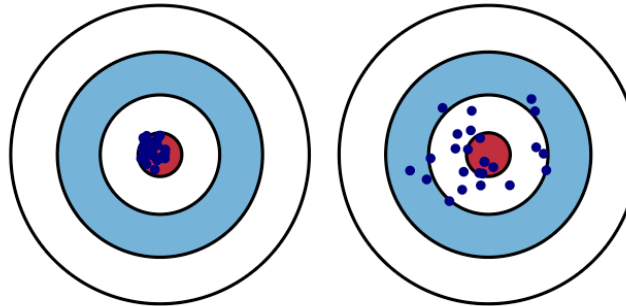
The Hong Kong University of Science and Technology (Guangzhou)

Recall of Last Lecture

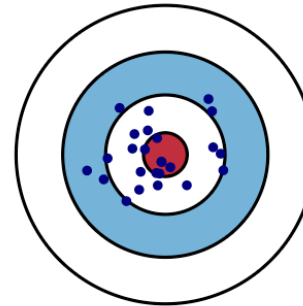
Bias and Variance

- **Bias:** the difference between the average **prediction of model** and the **correct value** which we are trying to predict.
- **Variance:** Model with high variance pays a lot of attention to **training data** and does not generalize on the **test data**.

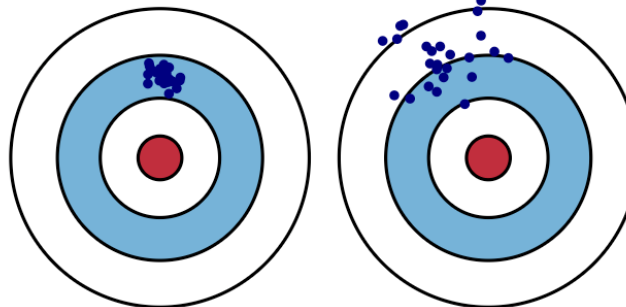
Low bias and
low variance



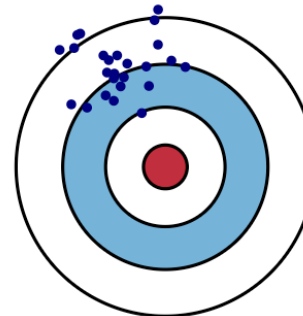
Low bias and
large variance



Large bias and
low variance



Large bias and
large variance



Bias-Variance Plot

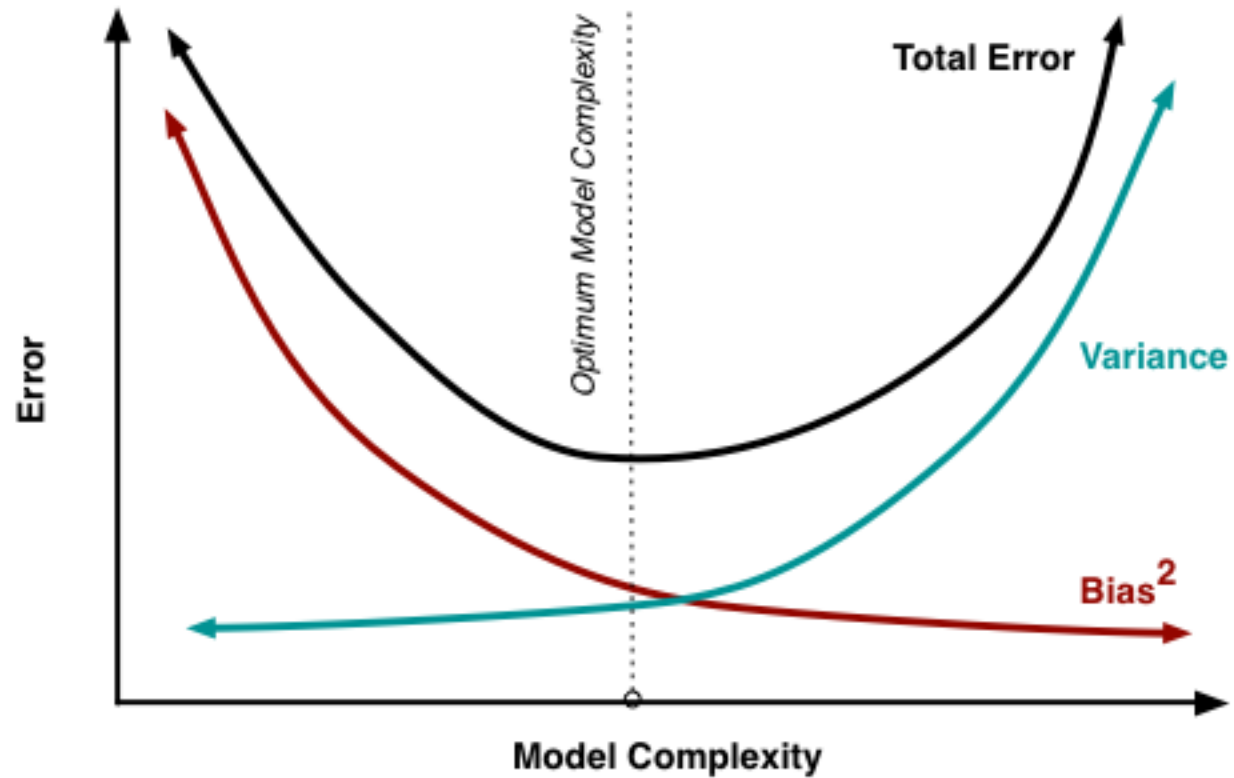
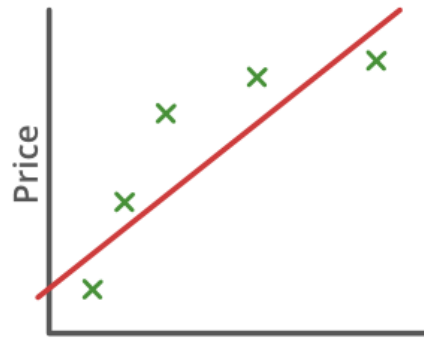
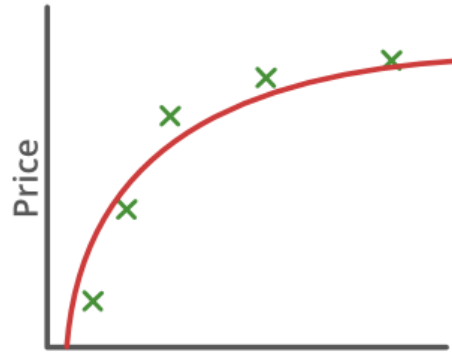


Image from <http://scott.fortmann-roe.com/docs/BiasVariance.html>

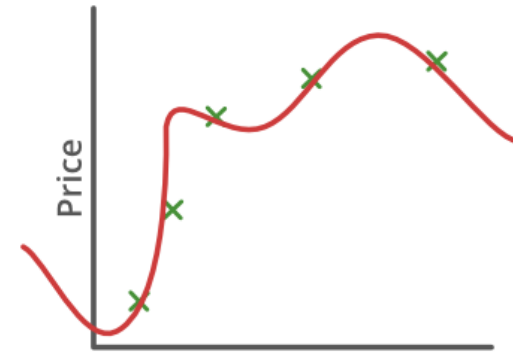
Overfitting and underfitting



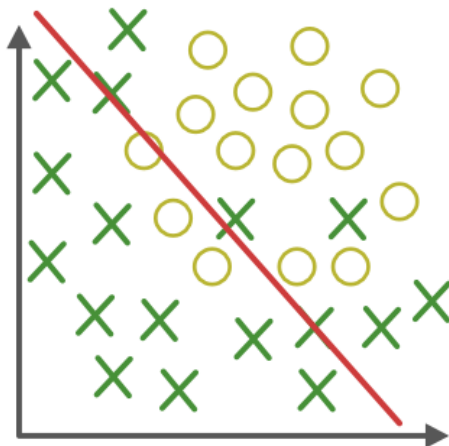
Size
 $\theta_0 + \theta_1 x$



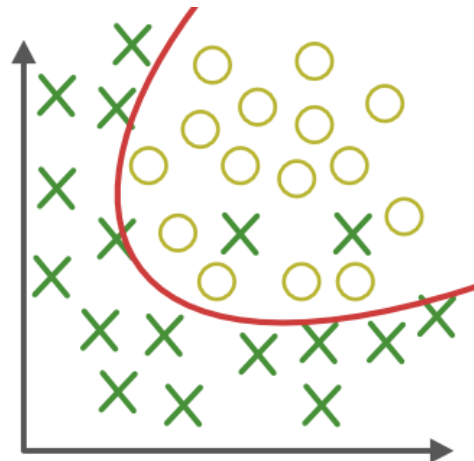
Size
 $\theta_0 + \theta_1 x + \theta_2 x^2$



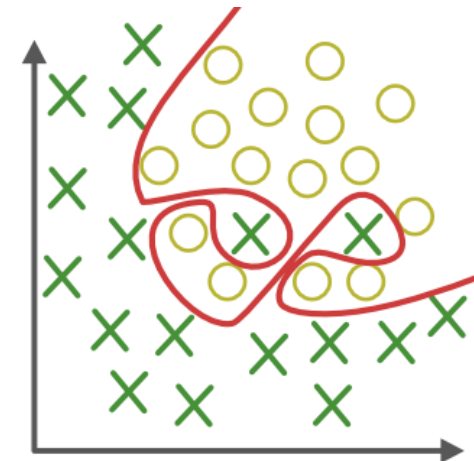
Size
 $\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$



Under-fitting
(too simple to
explain the variance)

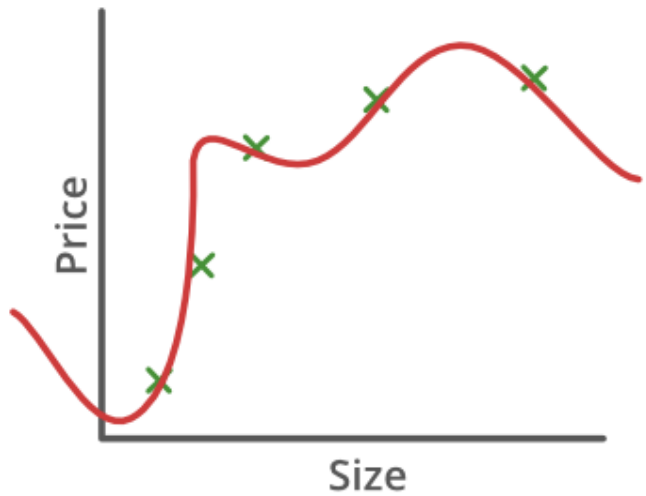


Appropriate-fitting

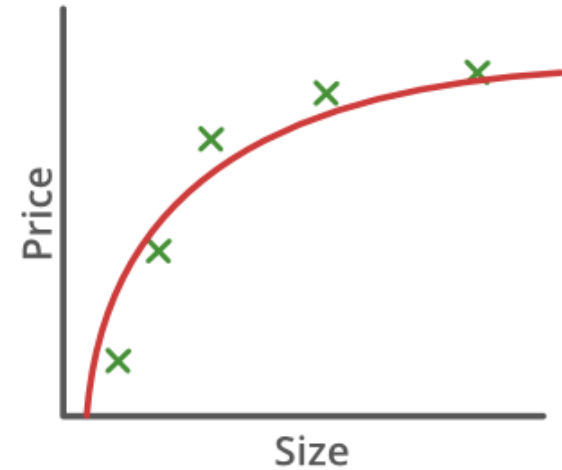


Over-fitting
(forcefitting--too
good to be true) $\mathfrak{G}\mathfrak{G}$

Intuition of Regularization



If we constrain θ_3 and θ_4 to small values



$$\theta_0 + \theta_1 x + \theta_2 x^2 + \cancel{\theta_3 x^3} + \cancel{\theta_4 x^4}$$

$$\approx \theta_0 + \theta_1 x + \theta_2 x^2$$

Common regularizers

How do we encourage small weights? or penalize large weights?

LASSO Regressor: penalize sum of the weight absolute values (L1 Norm)

$$\arg \min_{\mathbf{w}} L(\mathbf{w}) + \lambda \sum_i |w_i|$$

Ridge Regressor: penalize sum of the squared weights (L2 Norm)

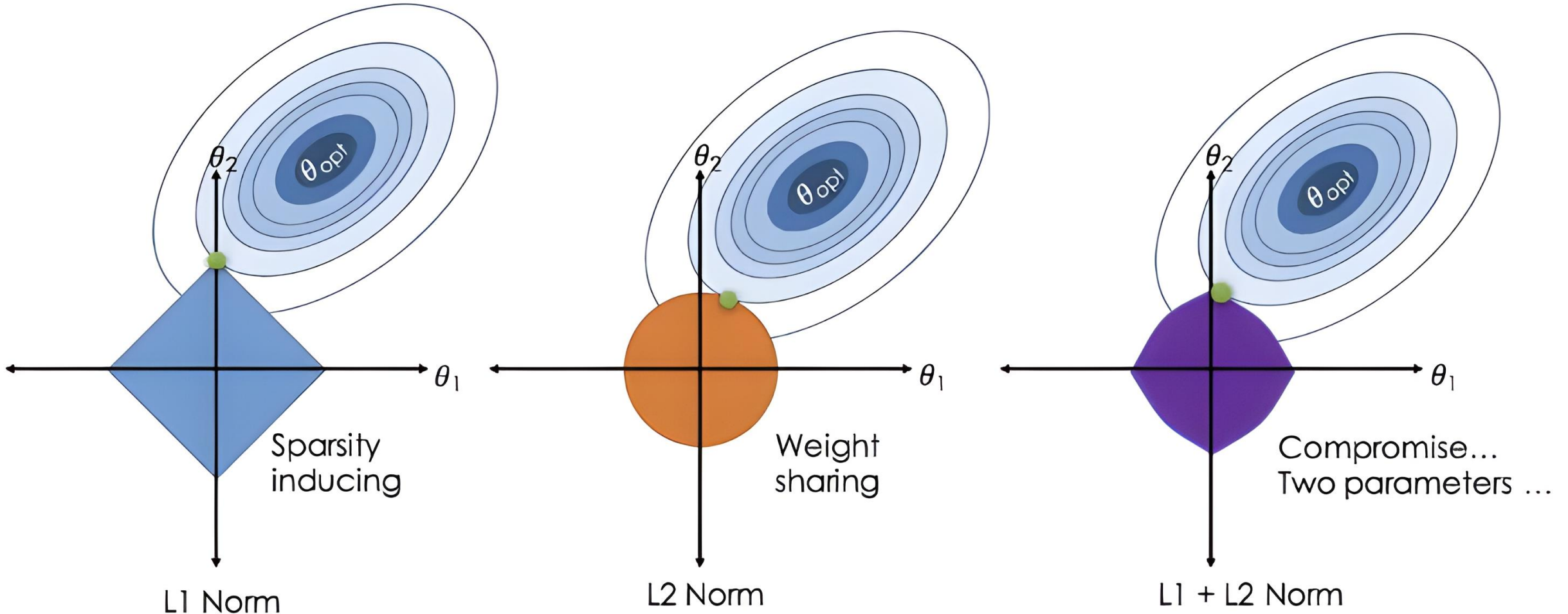
$$\arg \min_{\mathbf{w}} L(\mathbf{w}) + \lambda \sum_i |w_i|^2$$

λ : The **hyperparameter** to tune manually

LASSO lead to sparsity

L1 enforce sparse solutions:

The loss contours first hit the sharp corners of the L1 regularizer



Evaluation Metrics and Tuning

Confusion Matrix

		Predicted Class	
		Yes	No
Actual Class	Yes	TP	FN
	No	FP	TN

$$\text{Accuracy} = \frac{TP+TN}{n}$$

- TP: True Positive
- FP: False Positive
- FN: False Negative
- TN: True Negative

Classification Metrics

		Predicted Class	
		Yes	No
Actual Class	Yes	TP	FN
	No	FP	TN

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Accuracy} = \frac{TP + TN}{n}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

- Fraction of actual positives are predicted positive
- Fraction of predicted positives are actual positives

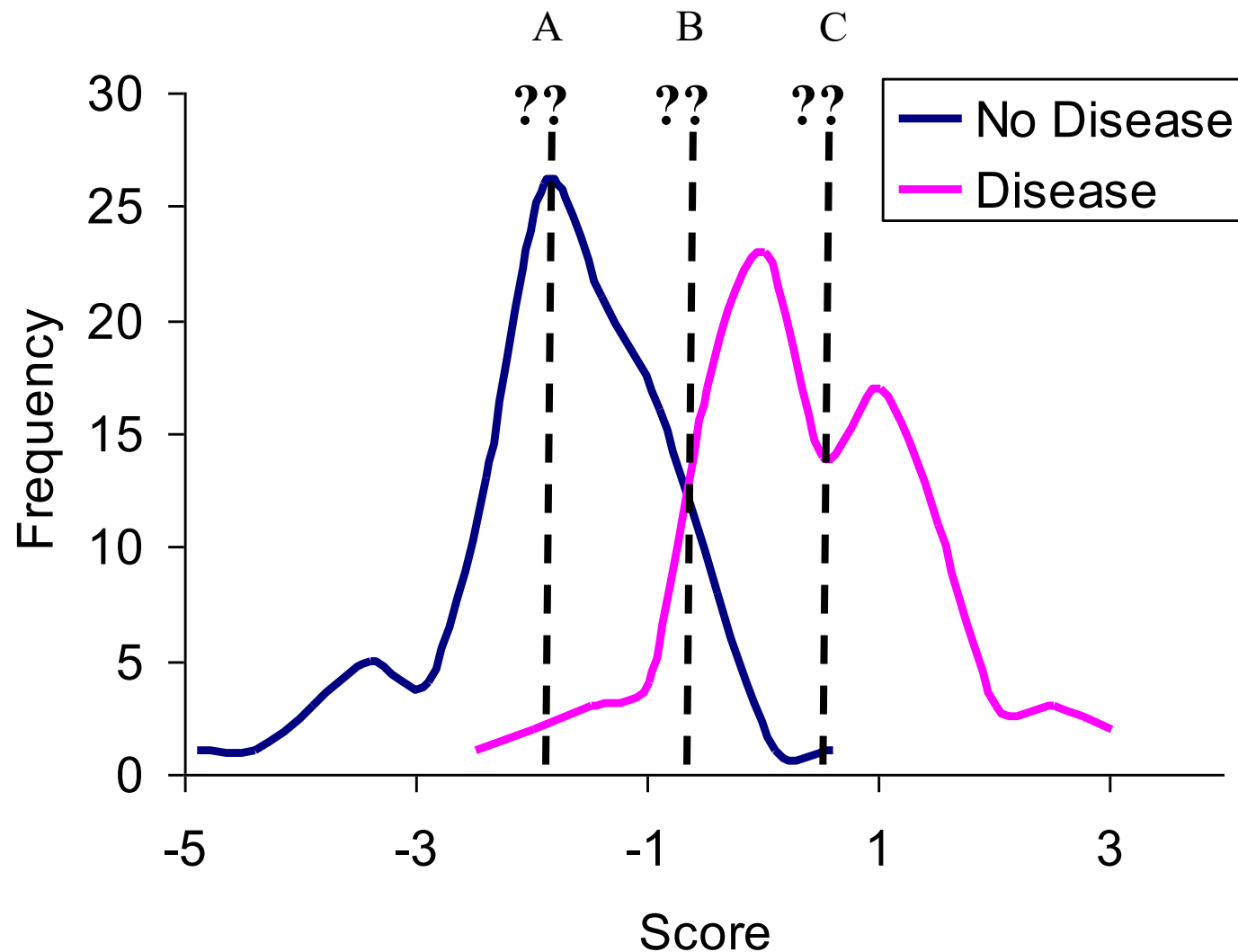
Classification Metrics

F1 score combines both precision and recall

$$\text{F1 Score} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Visual Perspective

The threshold of positive class can be tuned



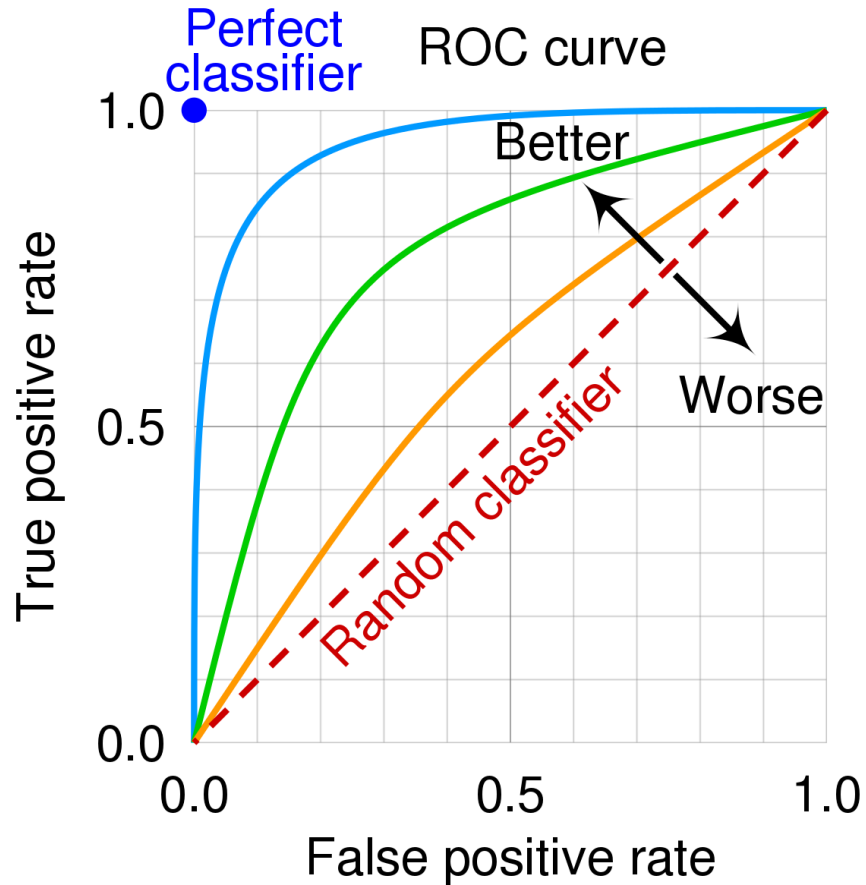
Which is the best one?



Scan this code to answer

ROC Curve

Tune the threshold of positive class can obtain the following ROC curve



$$\text{True Positive Rate} = \frac{TP}{TP + FN}$$

$$\text{False Positive Rate} = \frac{FP}{FP + TN}$$

ROC-AUC Score: Area Under the ROC curve

Example

Problem and Data (Breast cancer)

Diagnosis Result	Surgical pathology results		Total
	Patient (P)	Non-patient (N)	
Positive	80	100	180
Negative	20	800	820
Total	100	900	1000

Compute the True Positive Rate and False Positive Rate.



Scan this code to answer

Summary

- If the **True Positive Rate** of a diagnostic test is relatively low, there will be many false-negative patients. This will delay the patient's visit, affect the development and recovery of the disease, and even lead to premature death of the patient.
- If the **False Positive Rate** of a diagnostic test is relatively high, there will be many false positive patients. This will waste medical resources and cause unprovoked panic and anxiety among patients.

Hyper-parameter Tuning and Model Selection

Given Data:

Train data Z_{train}

Test data Z_{test}

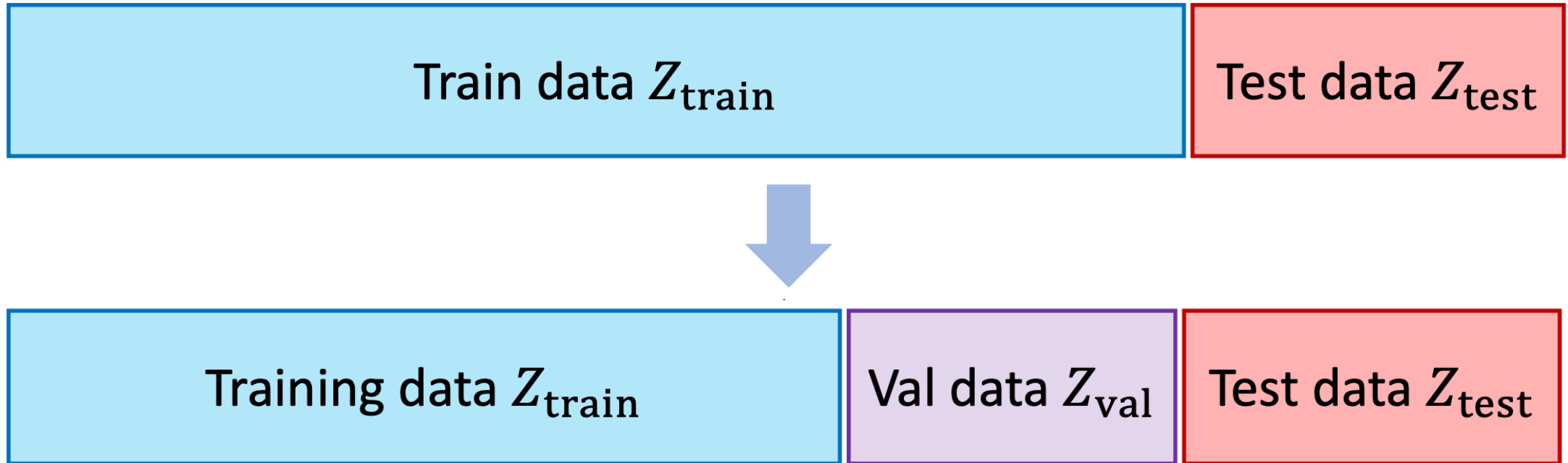
Goal: Choose hyperparameter/model for best generalization ability.

Can we just select hyperparameter that minimizes the test set Z_{test} ?

NO!!! This will lead to the overfitting on the test set.

- This will leakage information of test data
- Results on test data will not accurately reflect the model's generalization ability

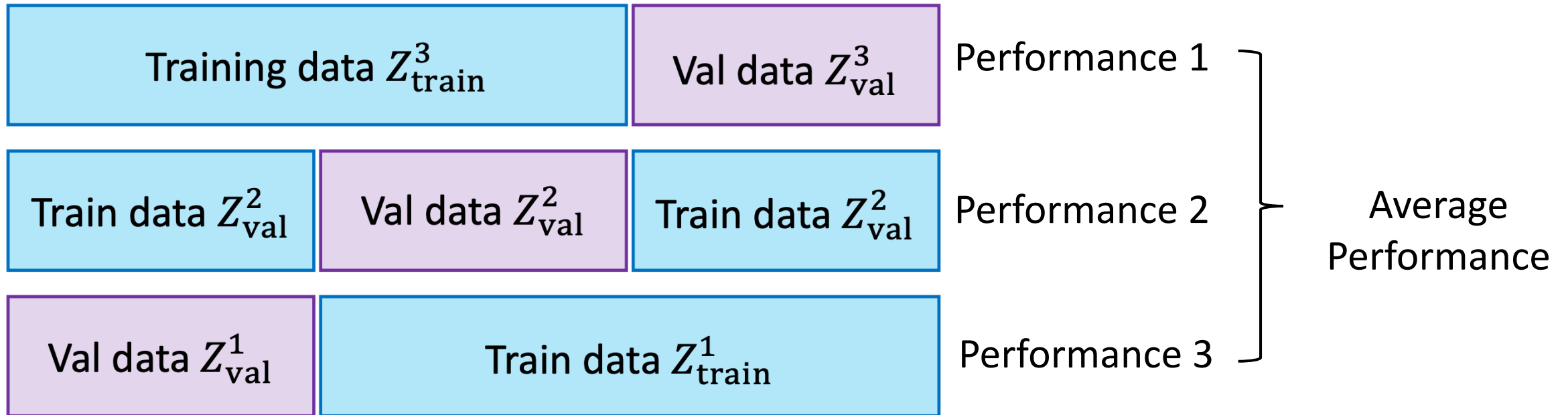
Training/Validation/Test Split



Solution: Tune hyperparameters/models on a **held-out validation data**

K-fold Cross Validation

An example of 3-fold



Tune model based on average performance
Avoid Overfitting on a selected validation set

An Example

Suppose we have a data sample with 6 observations:

1 [0.1, 0.2, 0.3, 0.4, 0.5, 0.6]

- The first step is to pick a value for k in order to determine the number of folds used to split the data.
- Here, we will use a value of $k = 3$. That means we will shuffle the data and then split the data into 3 groups.
- Because we have 6 observations, each group will have an equal number of 2 observations.

An Example

For example:

- 1 Fold1: [0.5, 0.2]
- 2 Fold2: [0.1, 0.3]
- 3 Fold3: [0.4, 0.6]

Three models are trained and evaluated with each fold given a chance to be the held out test set.

For example:

- **Model 1:** Trained on Fold1 + Fold2, Tested on Fold3
- **Model 2:** Trained on Fold2 + Fold3, Tested on Fold1
- **Model 3:** Trained on Fold1 + Fold3, Tested on Fold2

Taking the average of these three models as well as the *std* as the final result.

现在，我们请到了英伟达富有远见的首席执行官黄仁勋，
More now with Nvidia's visionary CEO, Jensen Huang,

Recall of last lecture

Classification Metrics

		Predicted Class	
		Yes	No
Actual Class	Yes	TP	FN
	No	FP	TN

$$\text{Precision} = \frac{TP}{TP + FP}$$

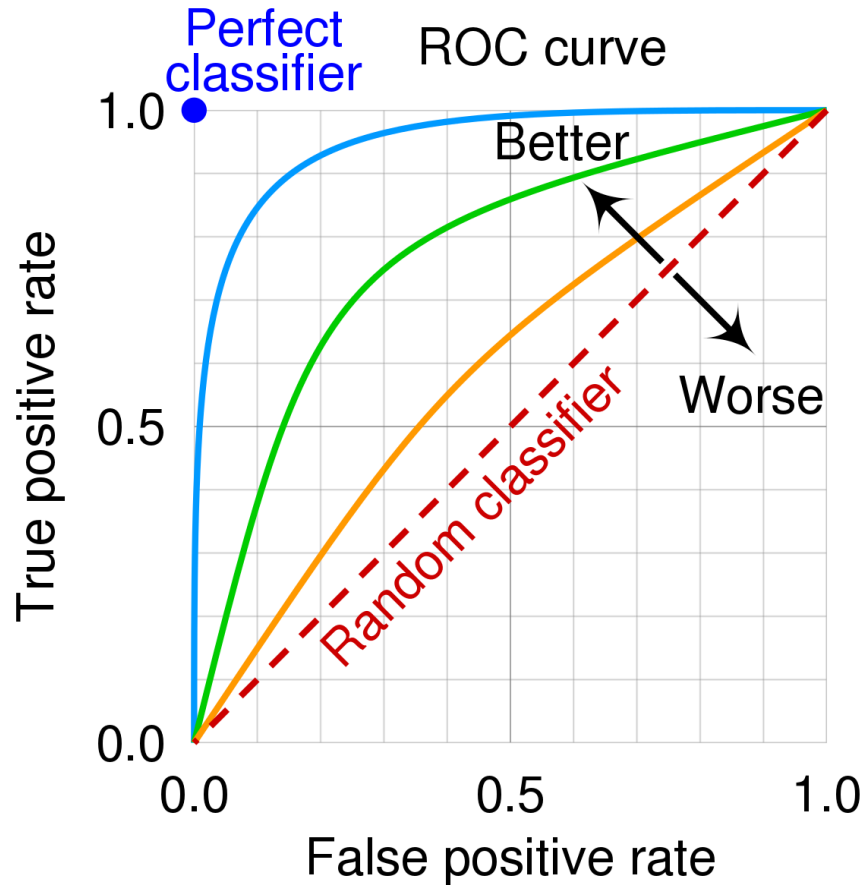
$$\text{Accuracy} = \frac{TP + TN}{n}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

- Fraction of actual positives are predicted positive
- Fraction of predicted positives are actual positives

ROC Curve

Tune the threshold of positive class can obtain the following ROC curve

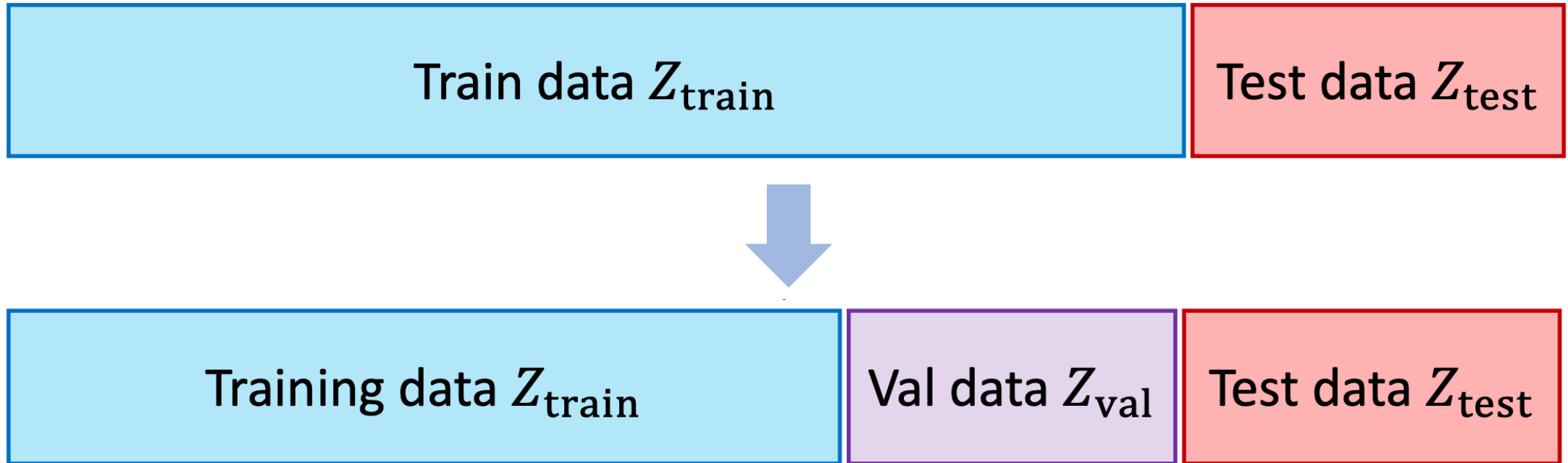


$$\text{True Positive Rate} = \frac{TP}{TP + FN}$$

$$\text{False Positive Rate} = \frac{FP}{FP + TN}$$

ROC-AUC Score: Area Under the ROC curve

Training/Validation/Test Split

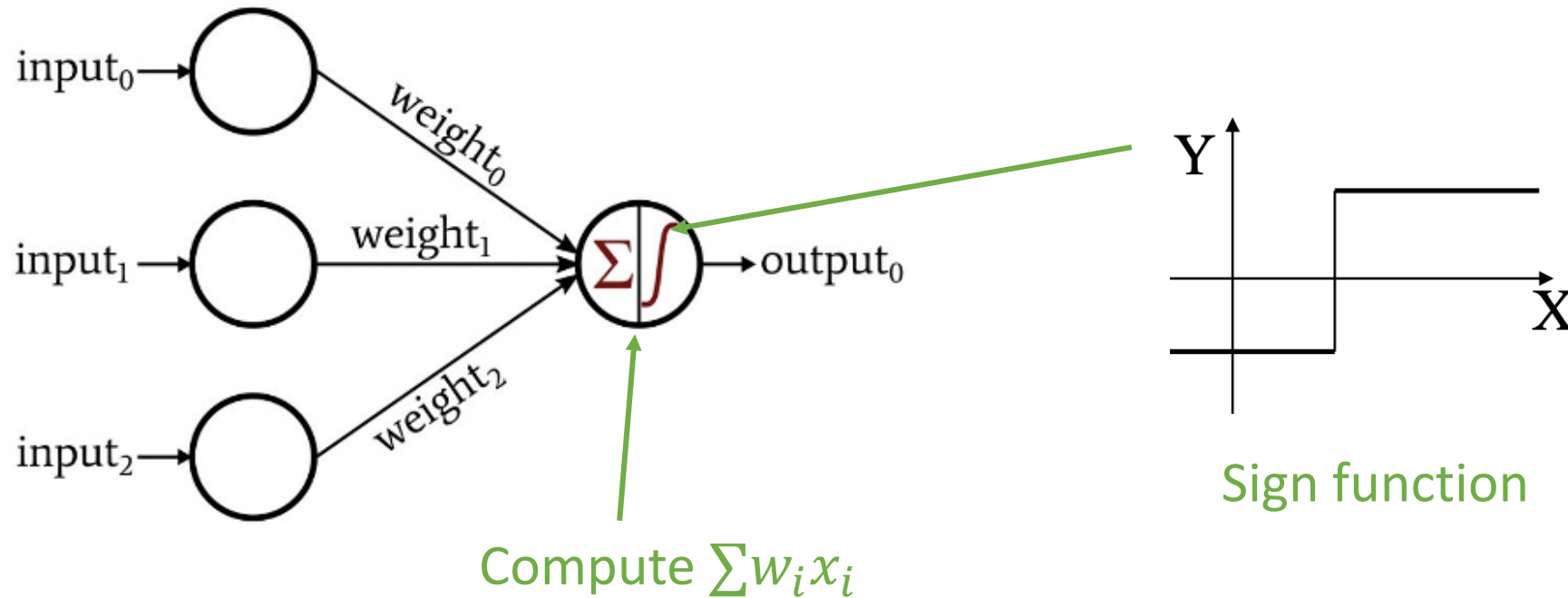


Solution: Tune hyperparameters/models on a **held-out validation data**

Multi-Layer Perception

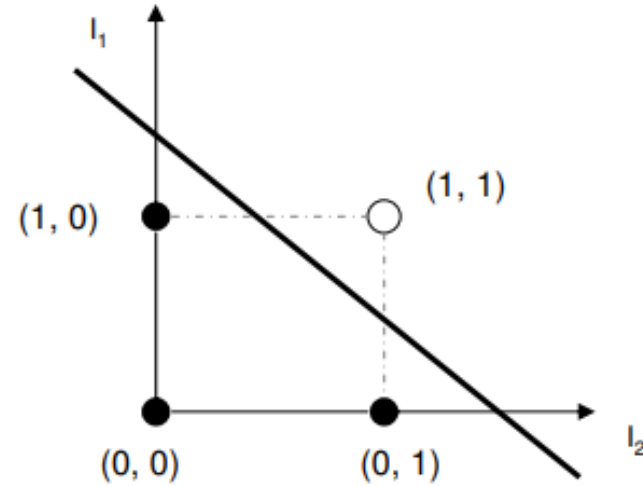
Perception

- Perceptron simulates 1 neuron
- Equation: $y = \text{sign}(\sum w_i x_i)$



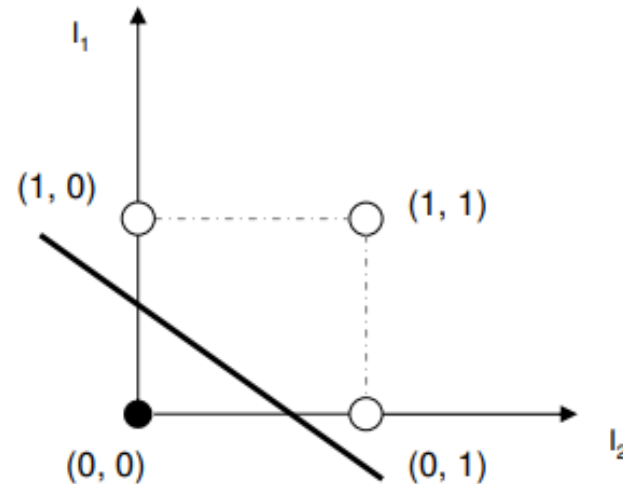
XOR Problem of Perception

AND		
I_1	I_2	out
0	0	0
0	1	0
1	0	0
1	1	1

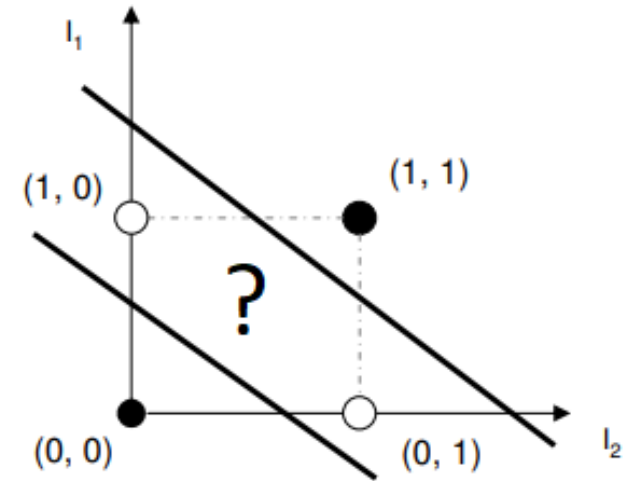


Perception cannot solve XOR problem

OR		
I_1	I_2	out
0	0	0
0	1	1
1	0	1
1	1	1



XOR		
I_1	I_2	out
0	0	0
0	1	1
1	0	1
1	1	0



Multi-Layer Perception

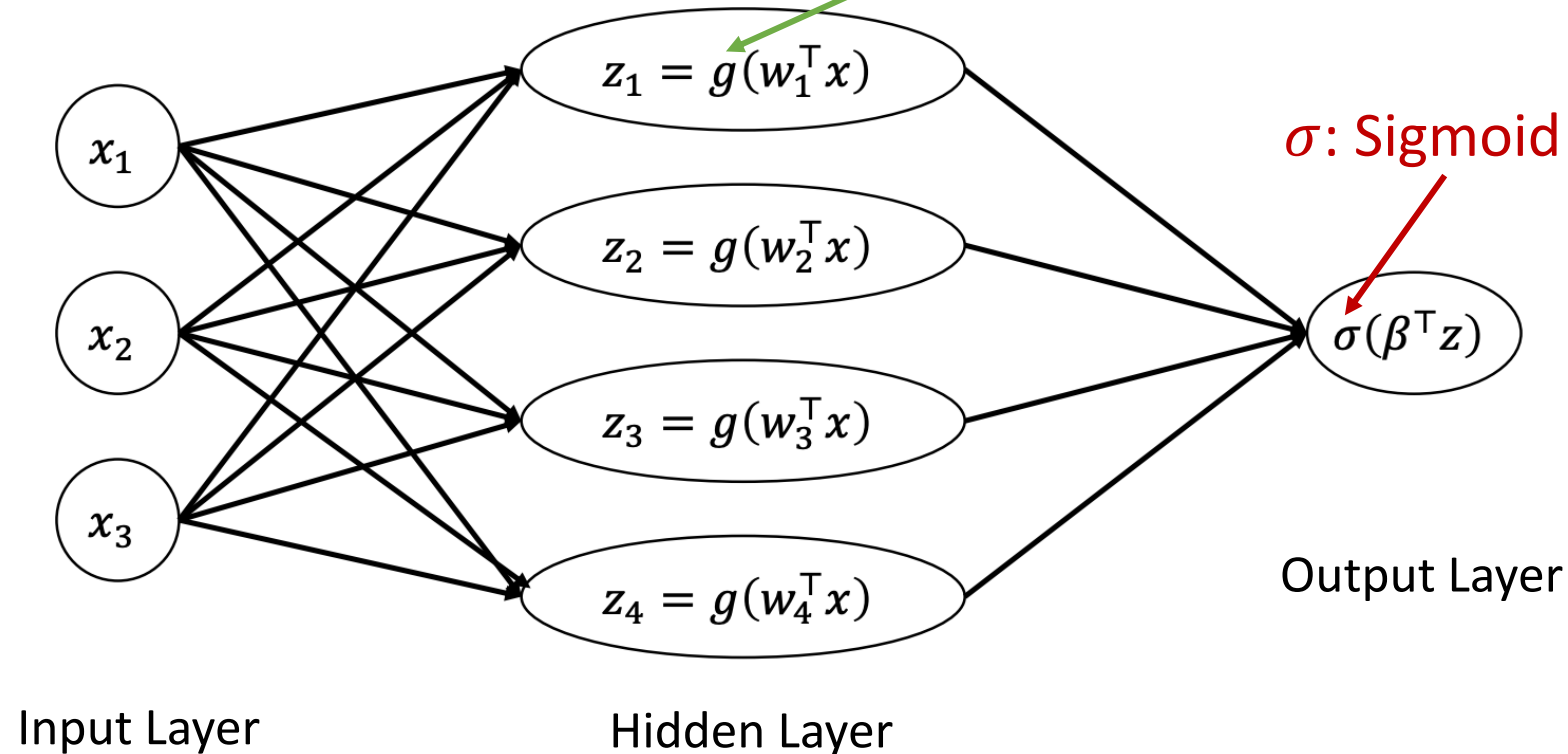
MLP is composed of multiple layers of neural networks

For binary classification: $f(\mathbf{x}) = \sigma(\beta^T g(\mathbf{W}\mathbf{x}))$

g : Non-linear activation function

σ : Sigmoid activation

MLP can stack many hidden layers

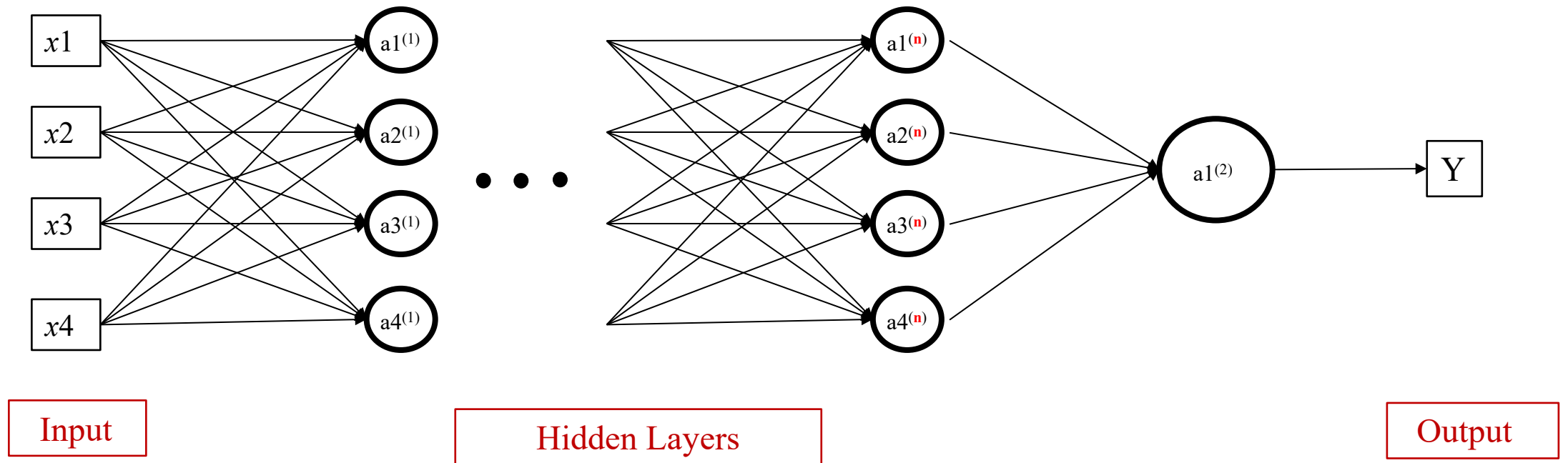


Universal Approximation by Neural Networks

[Hornik 1989] Regardless of the activation function, the dimension of the input space, a neural network with one hidden layer can approximate any continuous function

Deep Neural Network

DNN with n hidden layers



Softmax for Multi-Class Classification

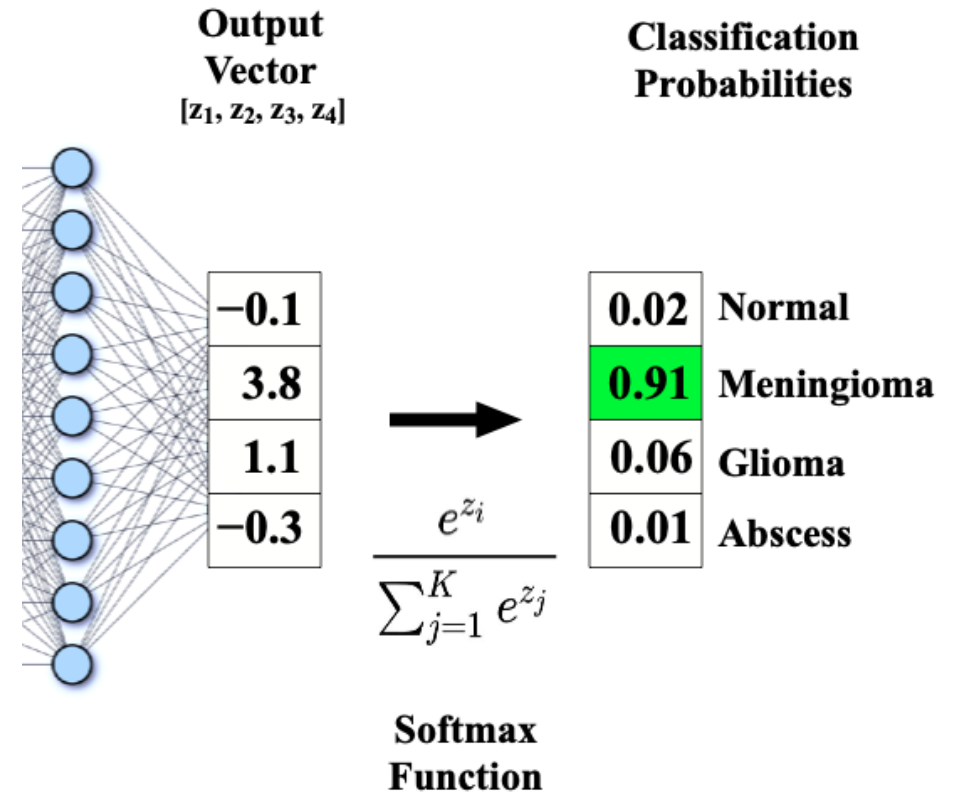
1. Probabilities are non-negative

Transfer $z_i \in (-\infty, +\infty)$ to non-negative by $\exp(z_i)$

2. Probabilities should sum to one ($\sum_i p(y = i|x) = 1$)

Achieve by normalizing

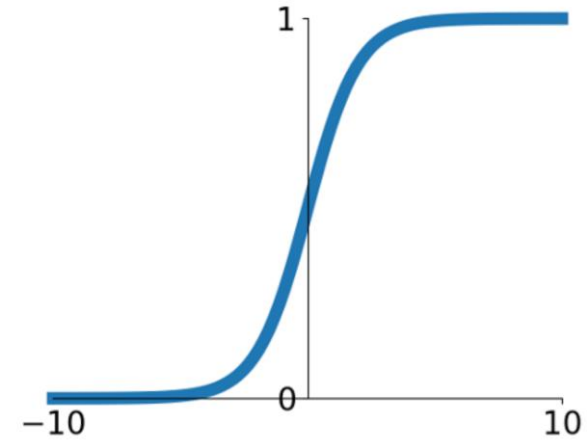
$$p(y = i|x) = f(x)_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$



Activation Functions

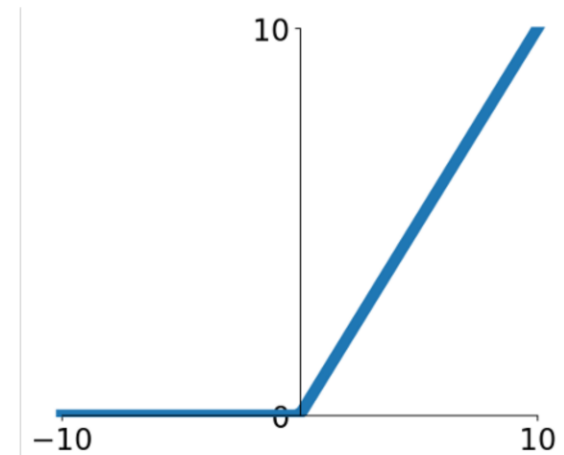
Sigmoid: $f(x) = \frac{1}{1+\exp(-x)}$

- Mostly used for **binary classification**
- **Not often** used in hidden layers now.



ReLU: $f(x) = \max(0, x)$

- Good default choice for hidden layers

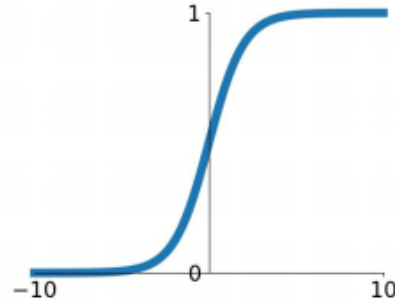


Activation functions

ReLU is a good default choice for most problems!

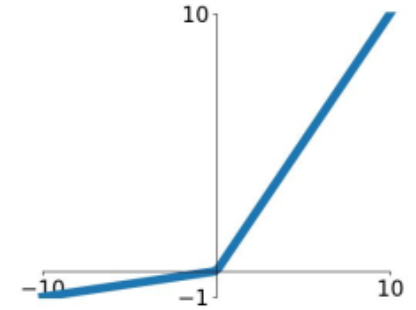
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



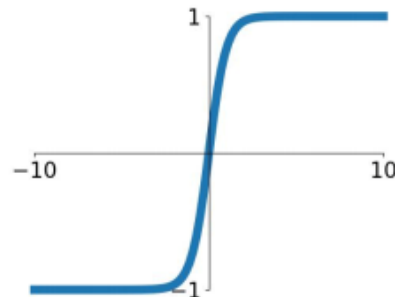
Leaky ReLU

$$\max(0.1x, x)$$



tanh

$$\tanh(x)$$

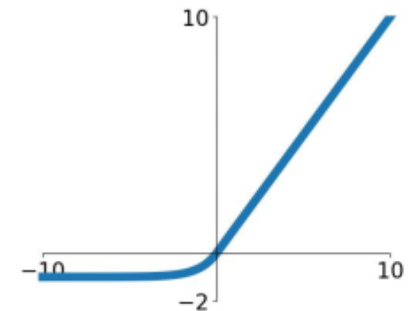


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

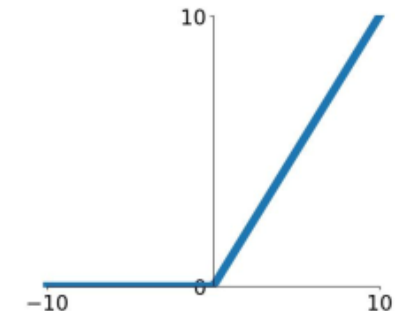
ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



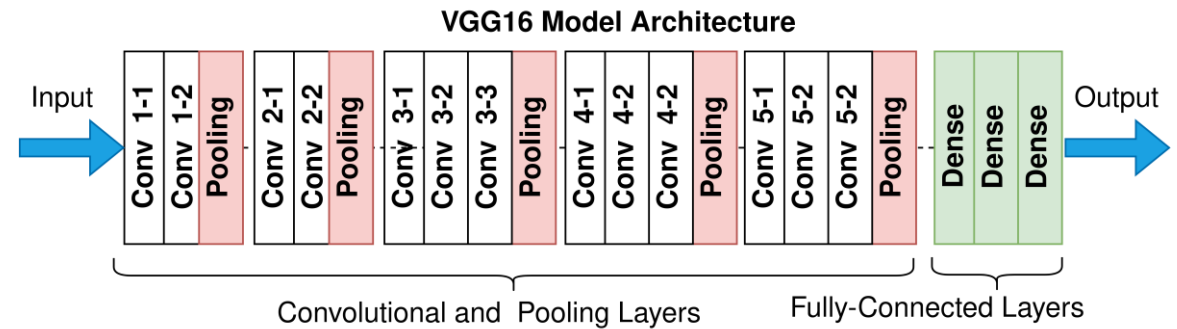
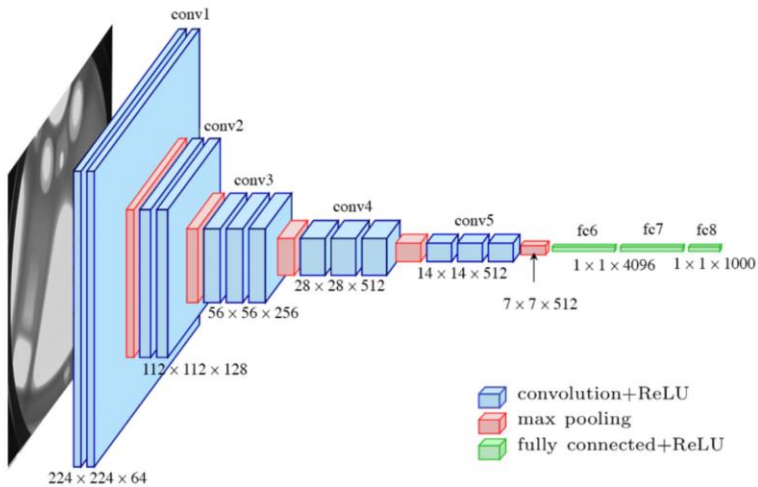
ReLU

$$\max(0, x)$$



Deeper or Wider?

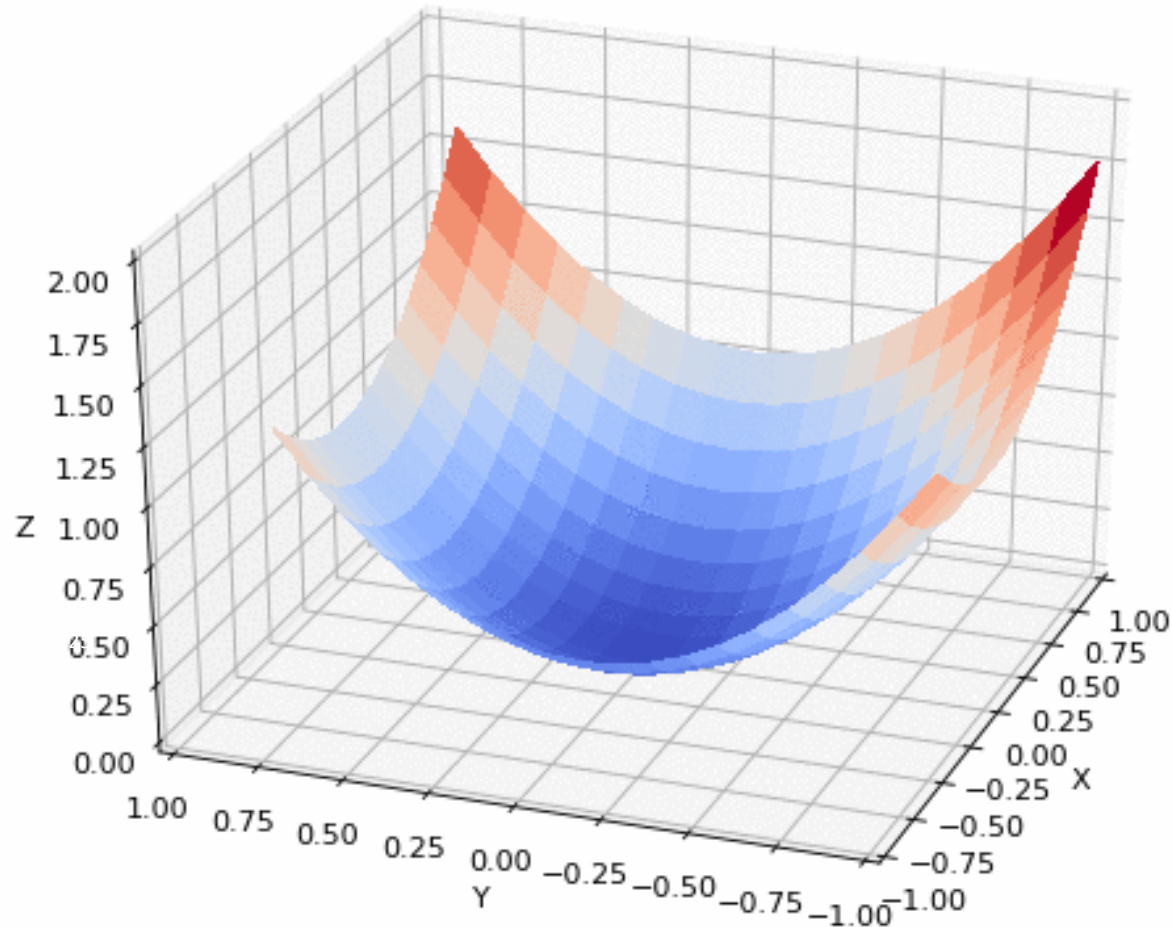
- Early researchers focus on the width of neural networks
- But now, we are in the age of deep learning



VGG is a classical deep convolutional neural network

Optimization

Gradient descent



Idea: follow the direction that leads to steepest loss reduction.

$$\mathbf{W} \leftarrow \mathbf{W} - \lambda \frac{\partial L}{\partial \mathbf{W}}$$

Step size

Gradient

Gradient descent

$$\mathbf{W} \leftarrow \mathbf{W} - \lambda \frac{\partial L}{\partial \mathbf{W}} = \mathbf{W} - \lambda \frac{\partial \frac{1}{N} \sum_{i=1}^N L_i}{\partial \mathbf{W}} = \mathbf{W} - \lambda \frac{1}{N} \sum_{i=1}^N \frac{\partial L_i}{\partial \mathbf{W}}$$

$$L_i = - \sum_{j=1}^K y_j \log(p_j)$$

- K is the #class, y is one-hot vector

Converting prediction score to probability



Probabilities
must be ≥ 0

Probabilities
must sum to 1

cat	3.2	exp	24.5	normalize	0.13
car	5.1		164.0		0.87
frog	-1.7		0.18		0.00

Softmax function

$$s = f(x_i, W)$$

$$e^s$$

$$\frac{e^{s_k}}{\sum_j e^{s_j}}$$

Loss function

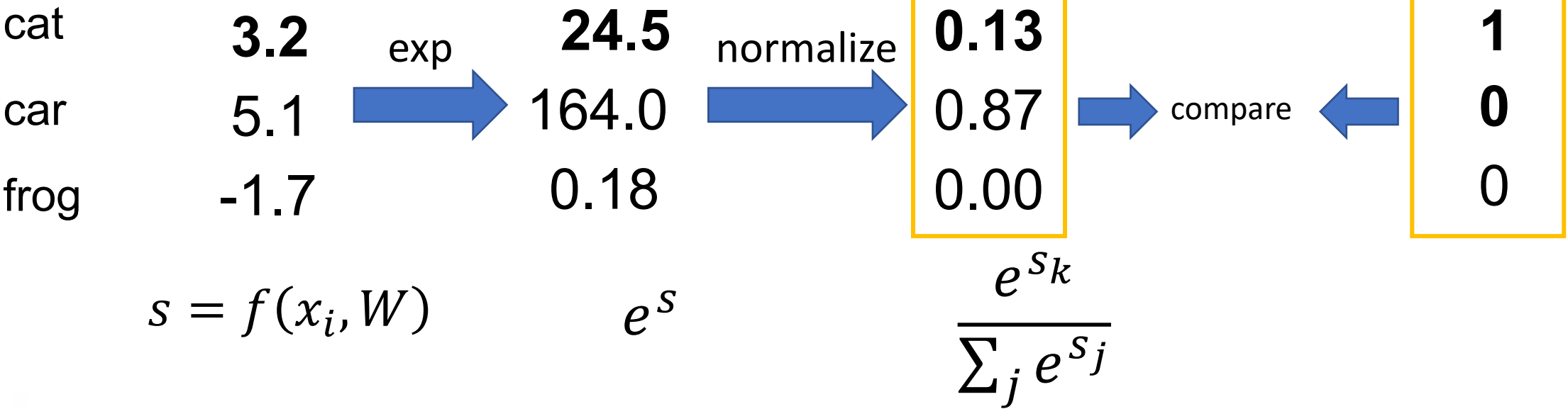
Comparing two probability distributions:
Cross entropy

$$L = -\frac{1}{N} \sum_i p_i \log(q_i) = -\frac{1}{N} \sum_i \log \left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}} \right)$$



Probabilities
must be ≥ 0

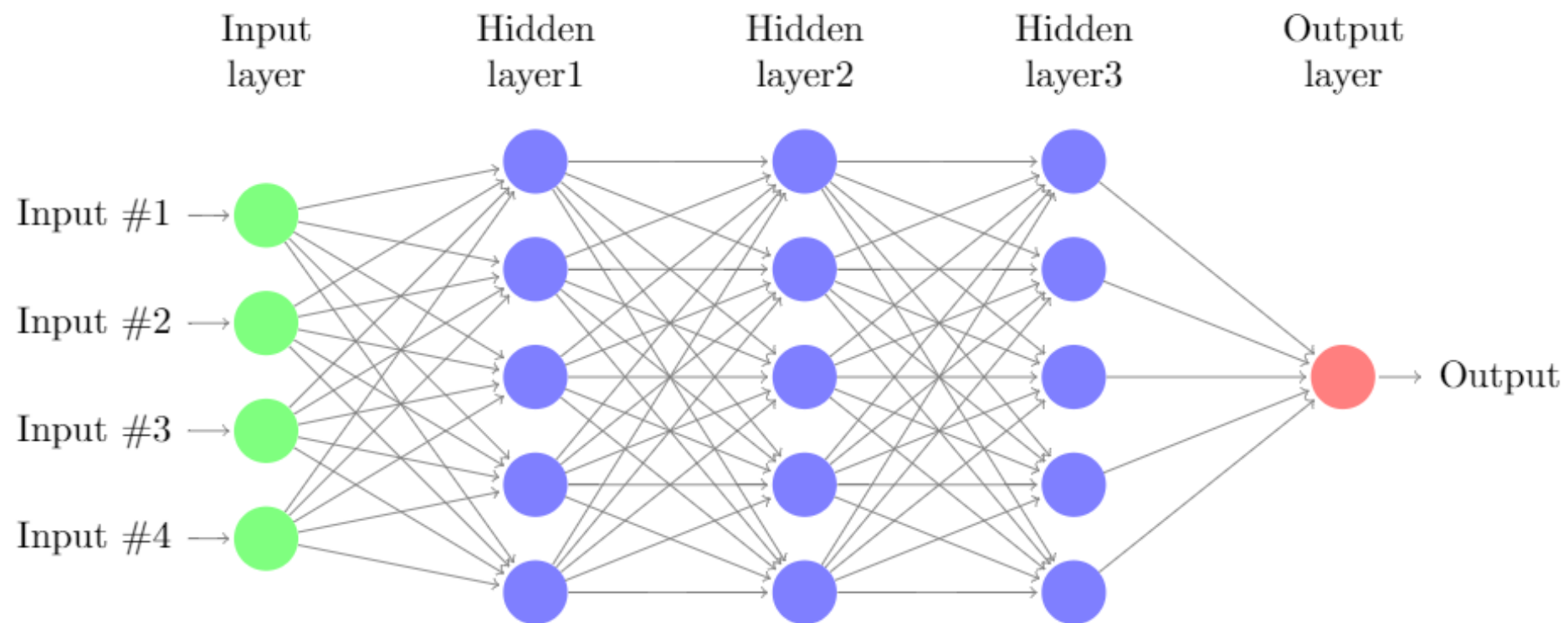
Probabilities
must sum to 1



How do we get the value of W and b ?

- Define a loss function
 - Softmax classifier
- Optimize W and b to minimize the loss function

Backpropagation: computing the gradient



Computation Graph

Forward

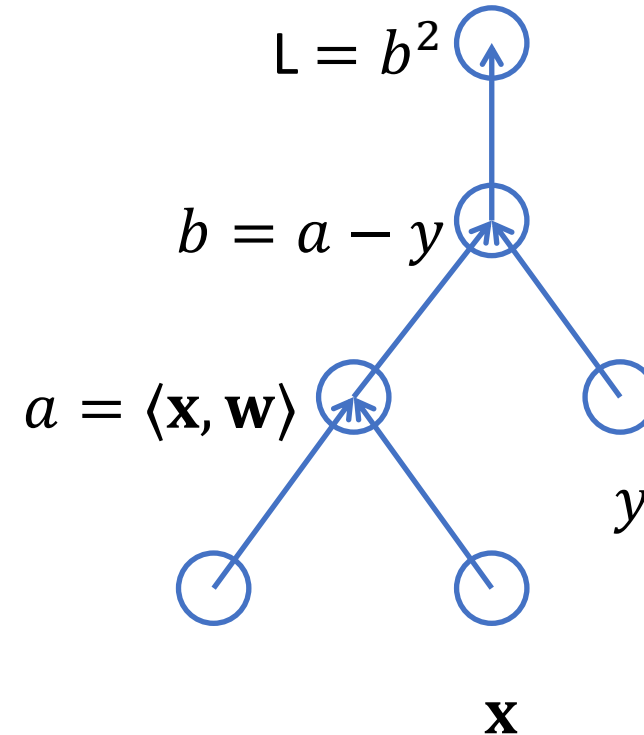
- Decompose into primitive operations
- Build a directed acyclic graph to present the computation

Backward

- Get the gradient by chain rule

$$\frac{\partial y}{\partial x} = \frac{\partial y}{\partial u_n} \frac{\partial u_n}{\partial u_{n-1}} \cdots \frac{\partial u_2}{\partial u_1} \frac{\partial u_1}{\partial x}$$

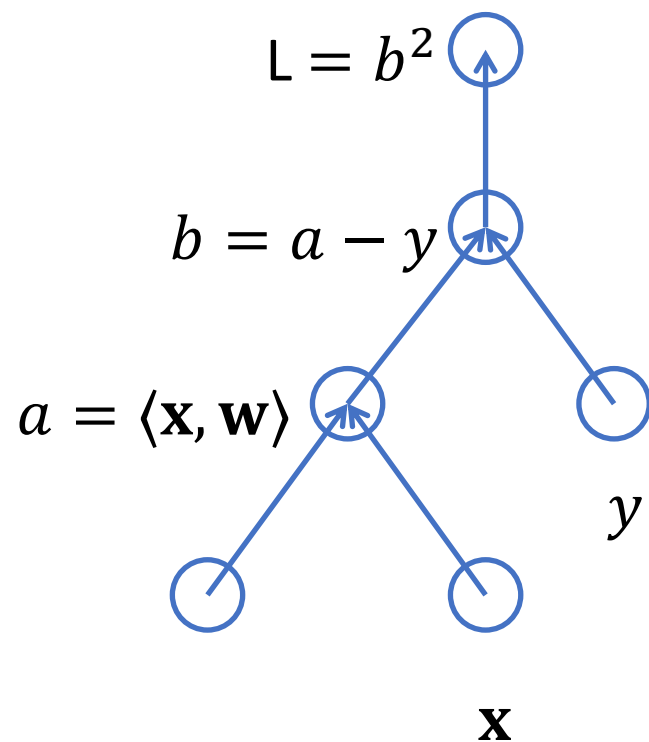
Assume $L = (\langle \mathbf{x}, \mathbf{w} \rangle - y)^2$



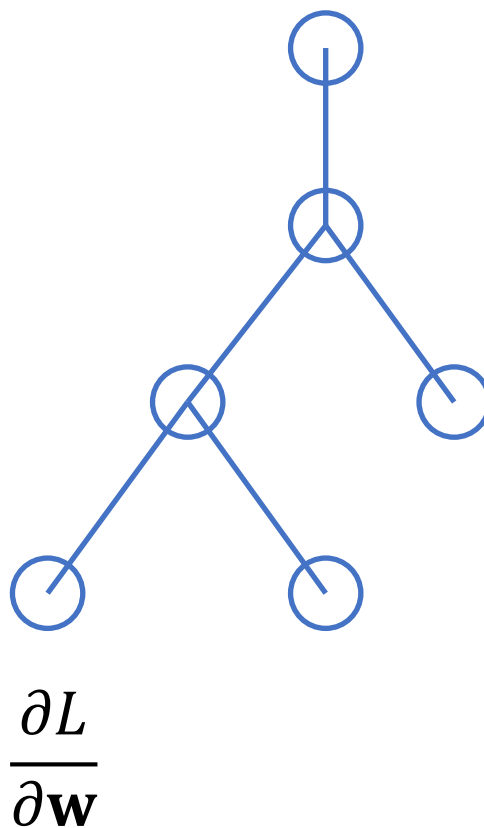
Reverse Accumulation

Assume $L = (\langle \mathbf{x}, \mathbf{w} \rangle - y)^2$

Forward



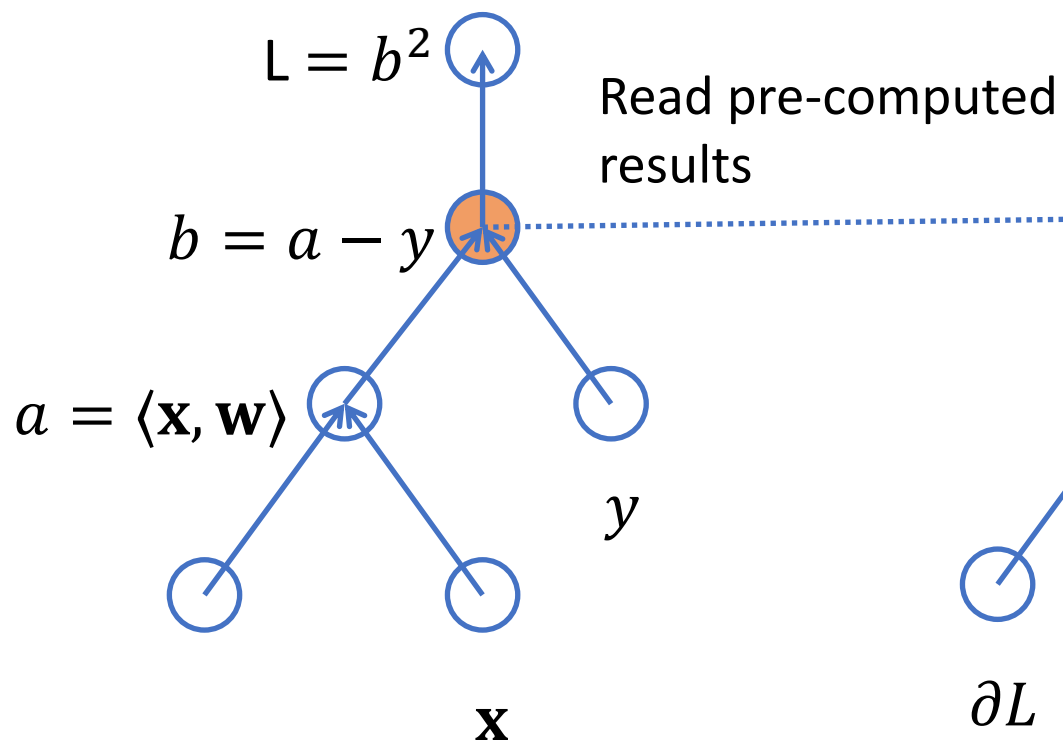
Backward



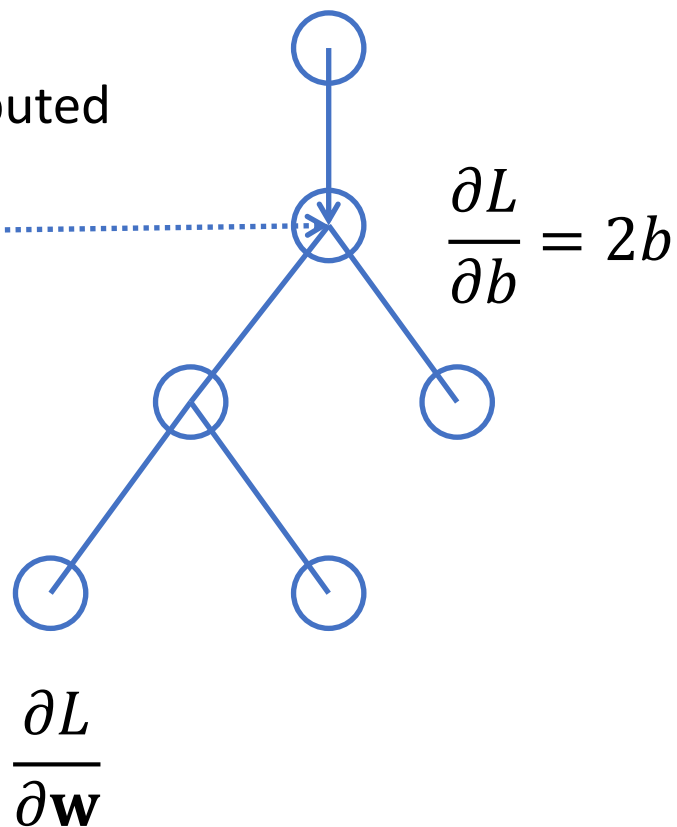
Reverse Accumulation

Assume $L = (\langle \mathbf{x}, \mathbf{w} \rangle - y)^2$

Forward



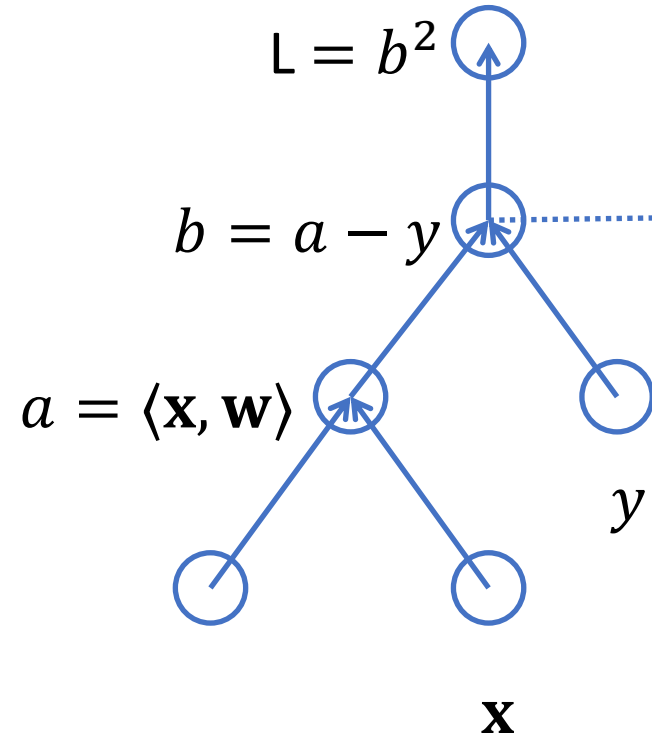
Backward



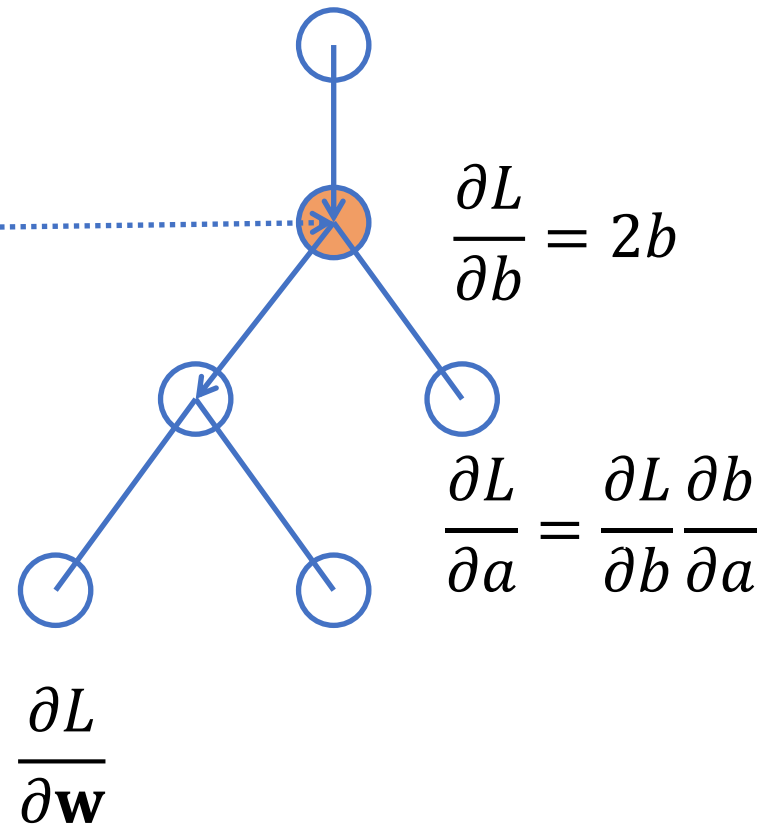
Reverse Accumulation

Assume $L = (\langle \mathbf{x}, \mathbf{w} \rangle - y)^2$

Forward



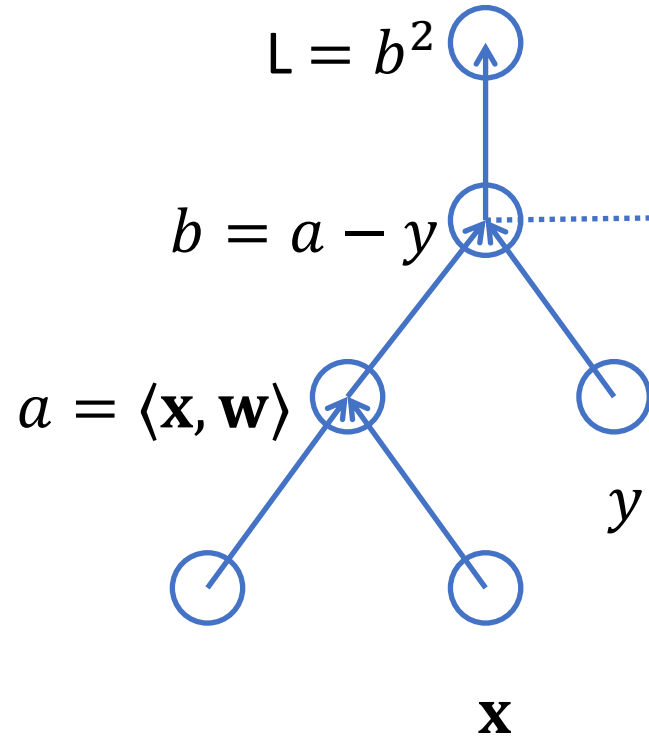
Backward



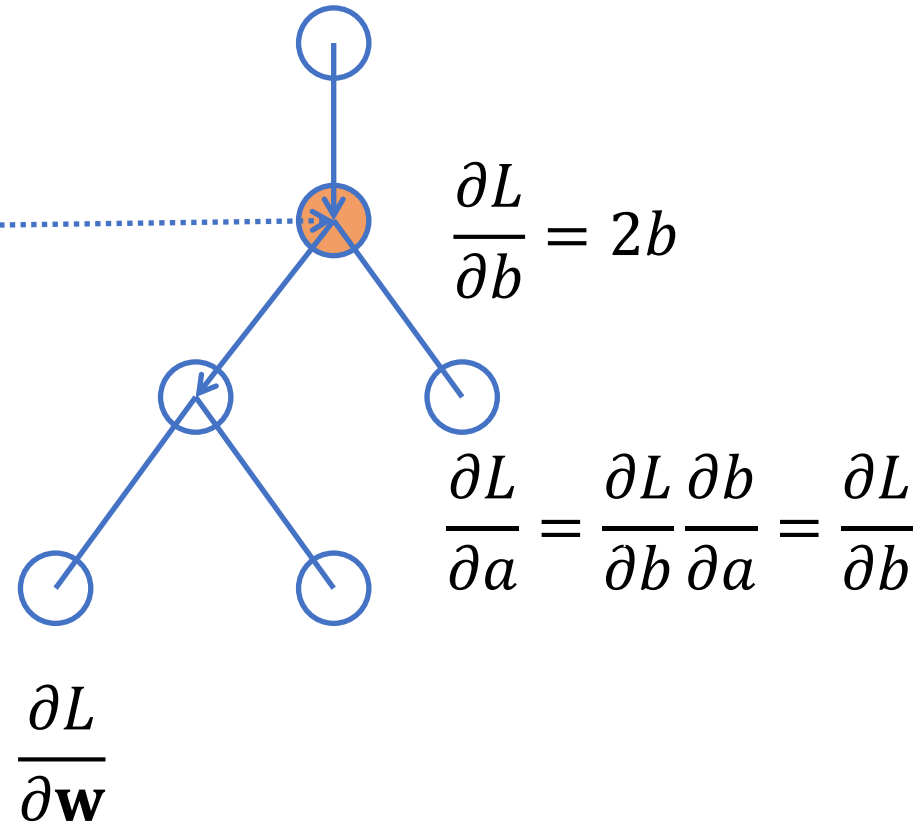
Reverse Accumulation

Assume $L = (\langle \mathbf{x}, \mathbf{w} \rangle - y)^2$

Forward

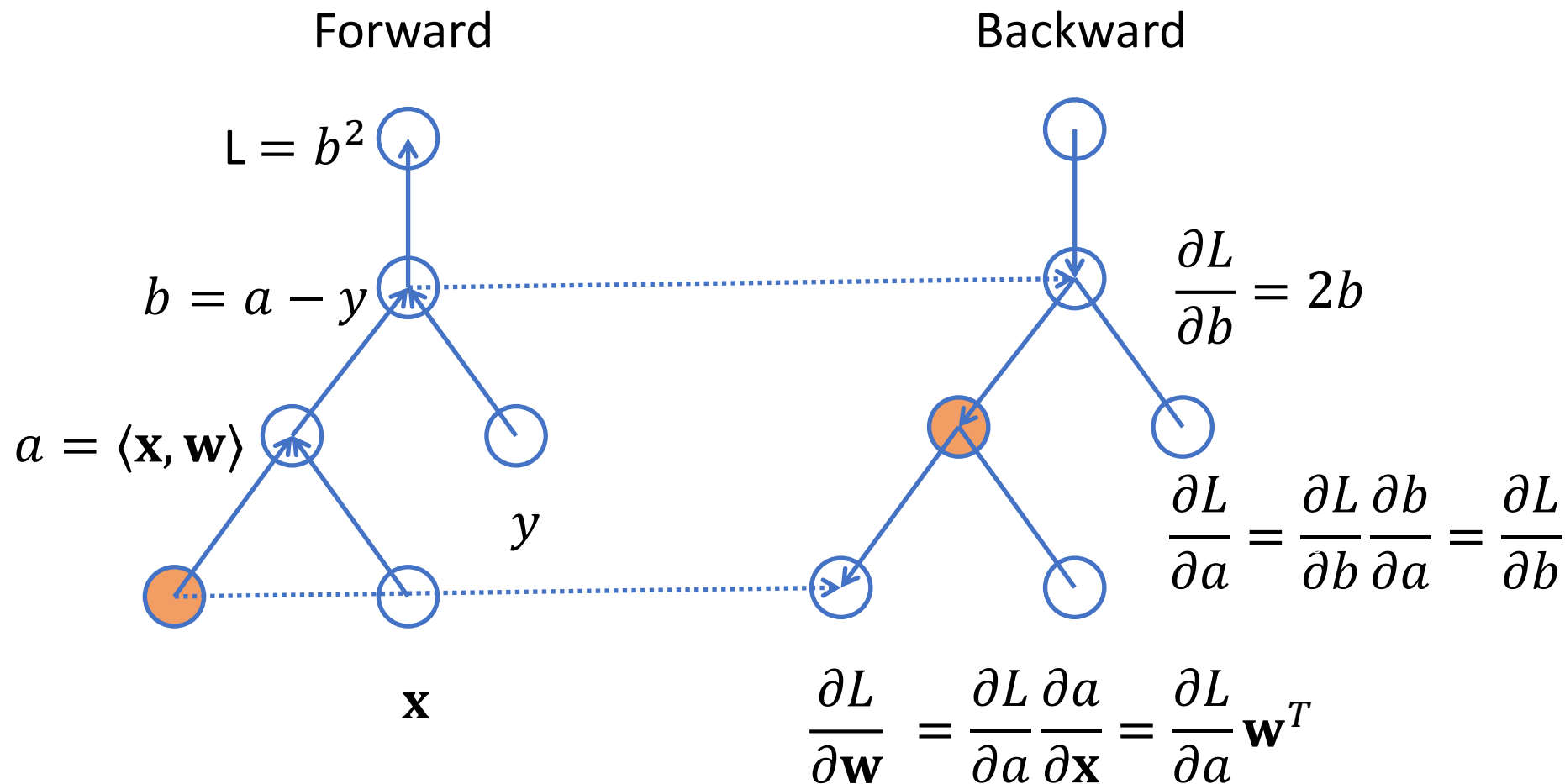


Backward



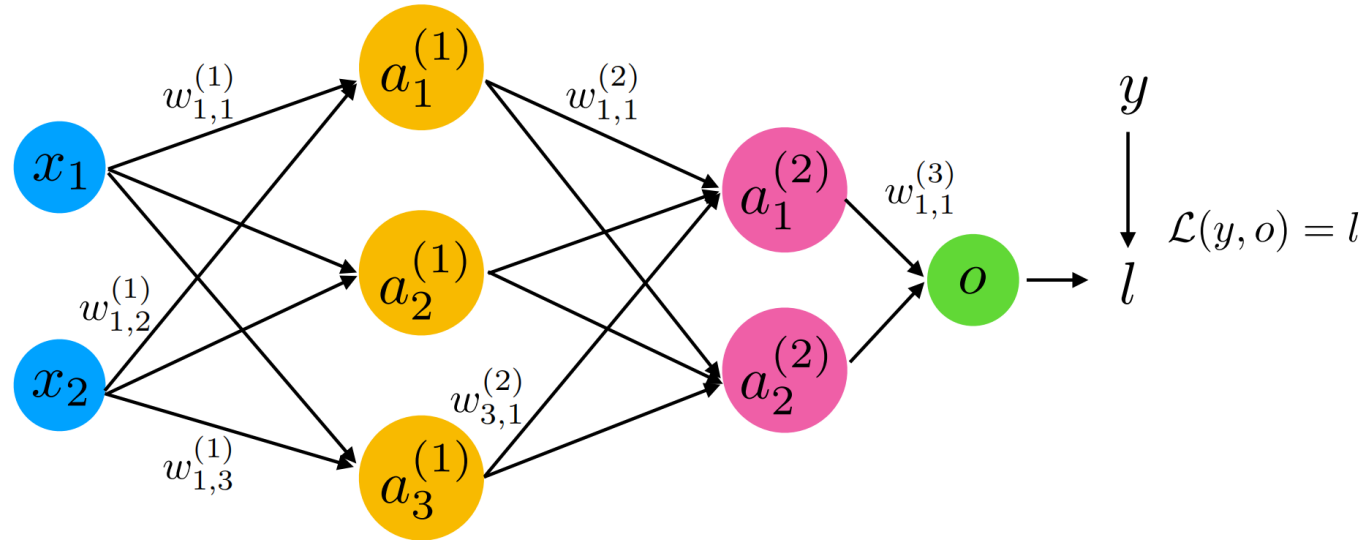
Reverse Accumulation

Assume $L = (\langle \mathbf{x}, \mathbf{w} \rangle - y)^2$



Deep Neural Networks are Difficult to Optimize!

Gradient Vanishing Problem



Suppose sigmoid activation function is applied:

$$\frac{\partial l}{\partial w_{1,1}^{(1)}} = \frac{\partial l}{\partial o} \cdot \frac{\partial o}{\partial a_1^{(2)}} \cdot \frac{\partial a_1^{(2)}}{\partial a_1^{(1)}} \cdot \frac{\partial a_1^{(1)}}{\partial w_{1,1}^{(1)}} + \frac{\partial l}{\partial o} \cdot \frac{\partial o}{\partial a_2^{(2)}} \cdot \frac{\partial a_2^{(2)}}{\partial a_1^{(1)}} \cdot \frac{\partial a_1^{(1)}}{\partial w_{1,1}^{(1)}}$$

$$a_1^{(2)} = \sigma(z_1^{(2)}) = \frac{1}{1 + \exp(-z_1^{(2)})}$$

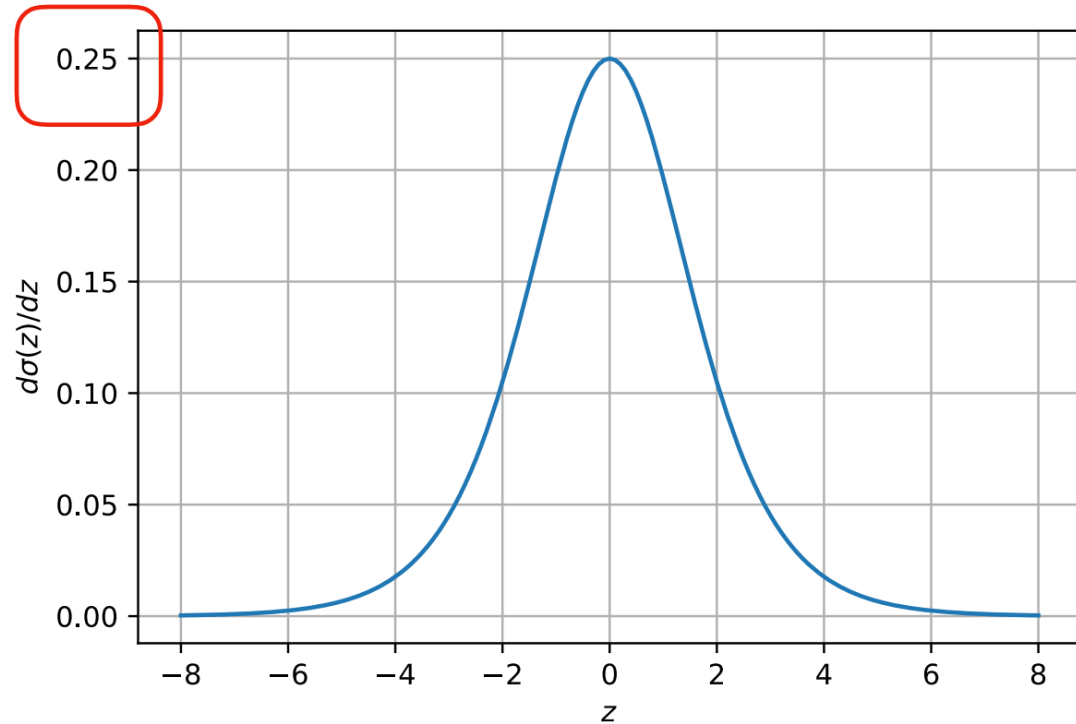
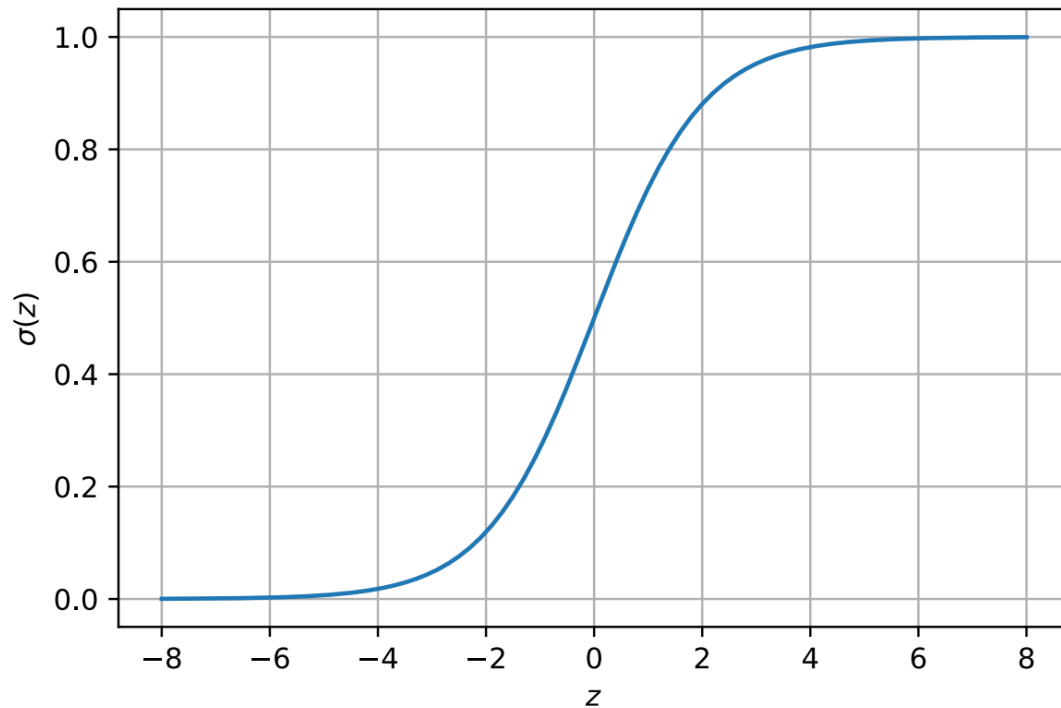
$$z_1^{(2)} = w_{1,1}^{(2)} a_1^{(1)} + w_{2,1}^{(2)} a_2^{(1)} + w_{3,1}^{(2)} a_3^{(1)}$$

Gradient Vanishing Problem

The highest gradient of sigmoid is **0.25**

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\frac{d}{dz}\sigma(z) = \frac{e^{-z}}{(1 + e^{-z})^2} = \sigma(z)(1 - \sigma(z))$$



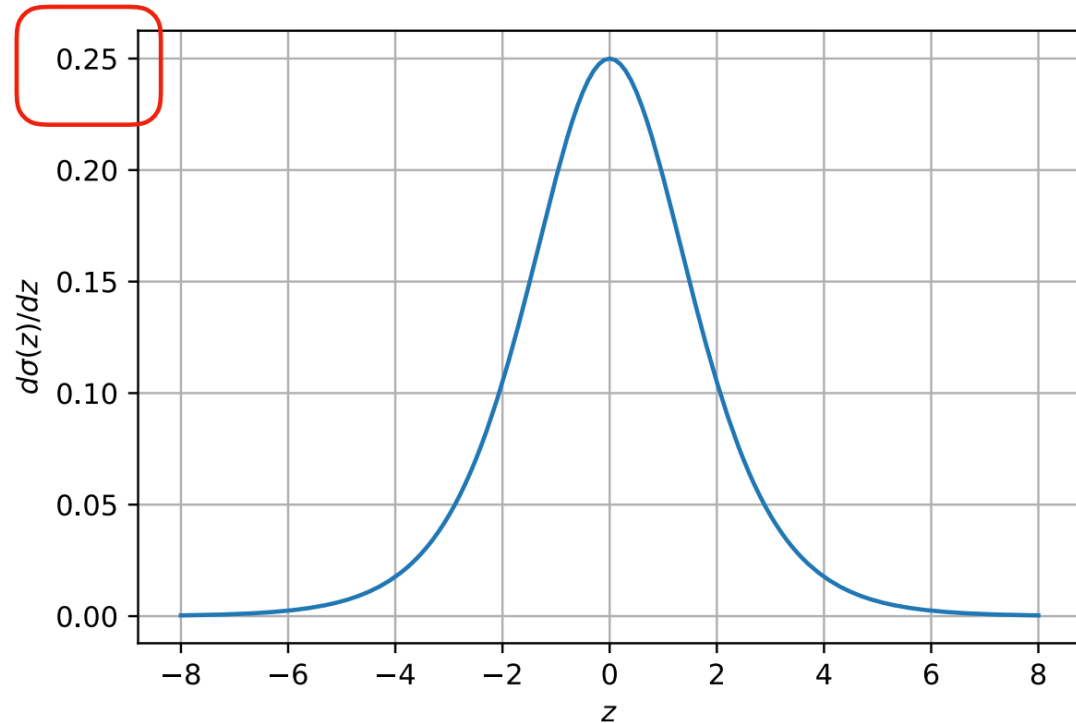
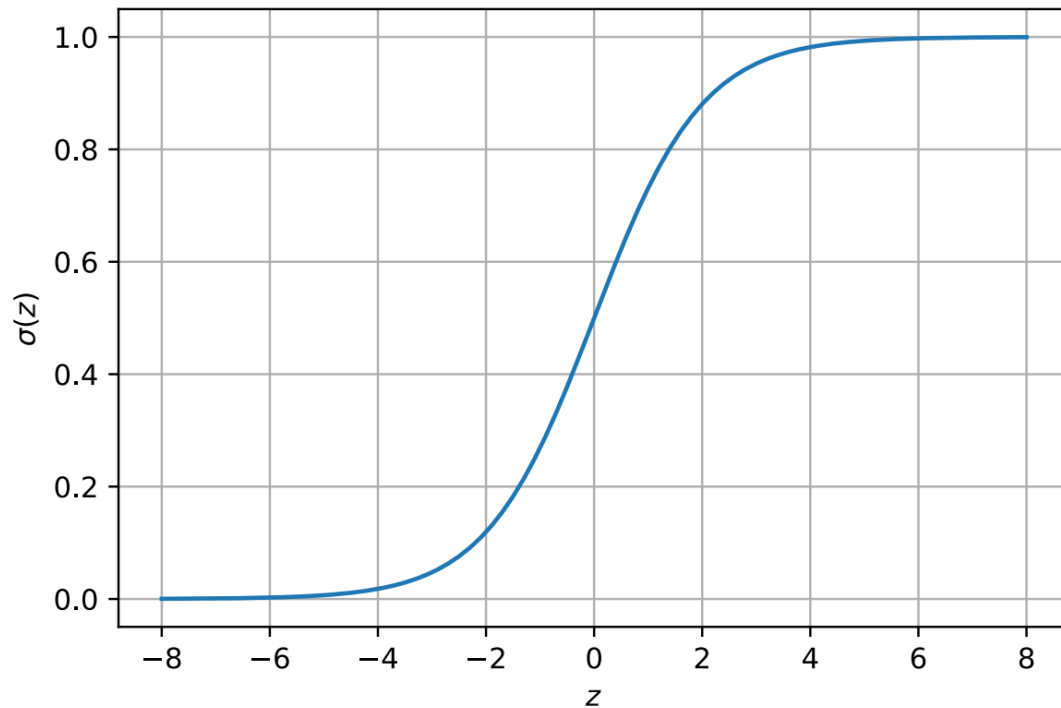
Gradient Vanishing Problem

The highest gradient of sigmoid is **0.25**

Suppose 10 layers using sigmoid is applied: $0.25^{10} \approx 10^{-6}$

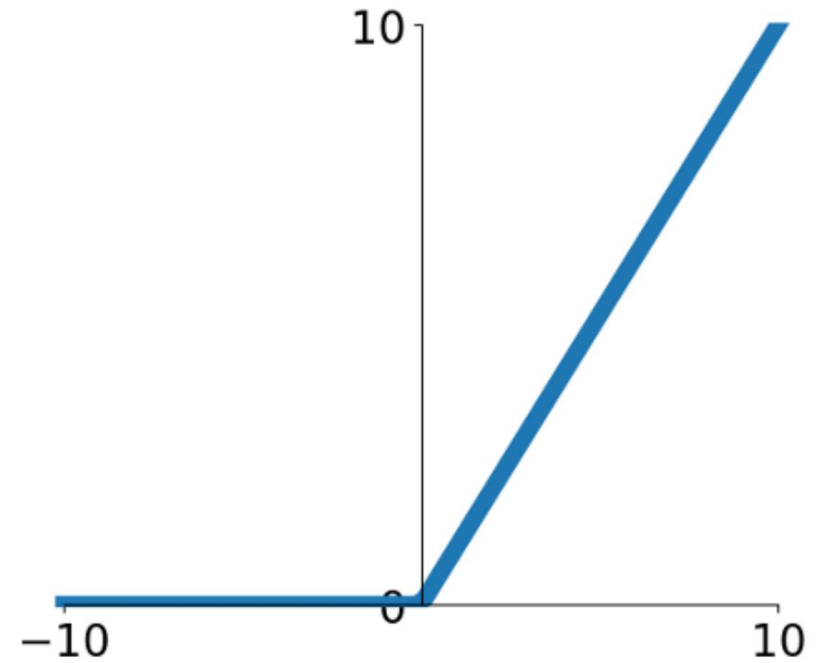
$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\frac{d}{dz}\sigma(z) = \frac{e^{-z}}{(1 + e^{-z})^2} = \sigma(z)(1 - \sigma(z))$$



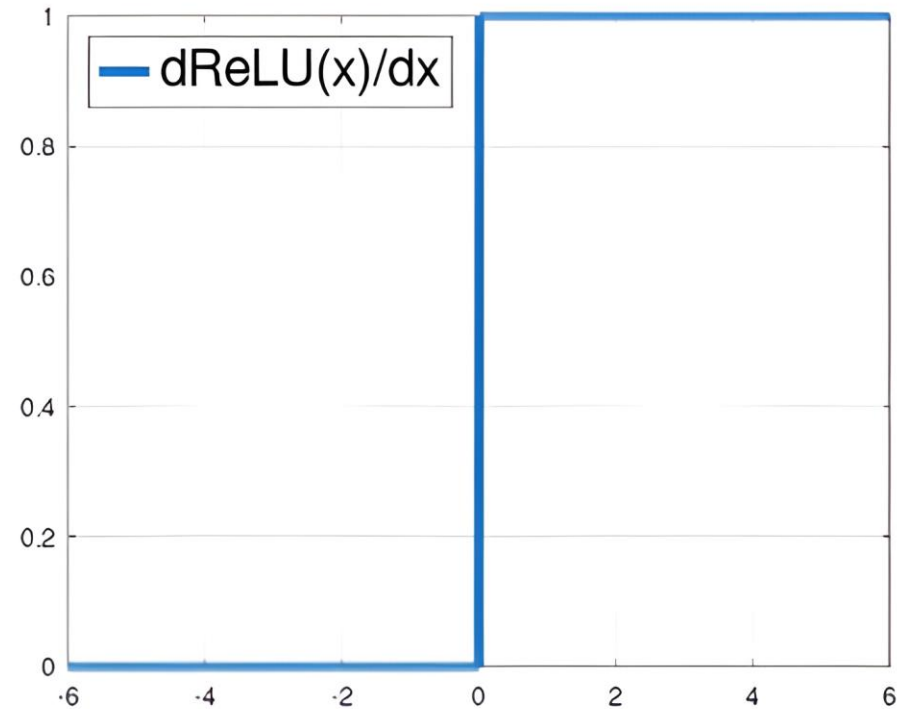
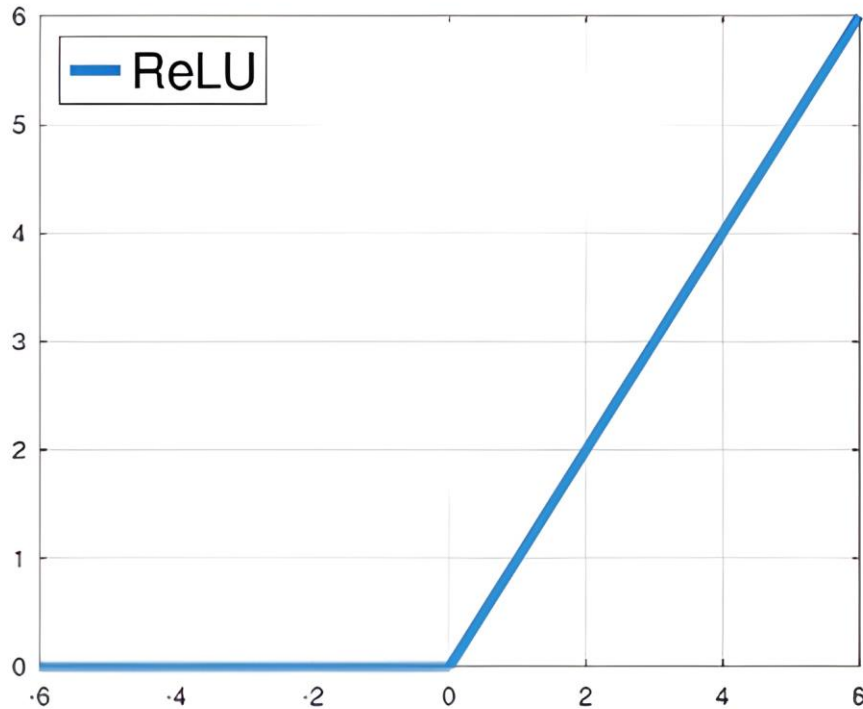
How about ReLU?

ReLU: $f(x) = \max(0, x)$



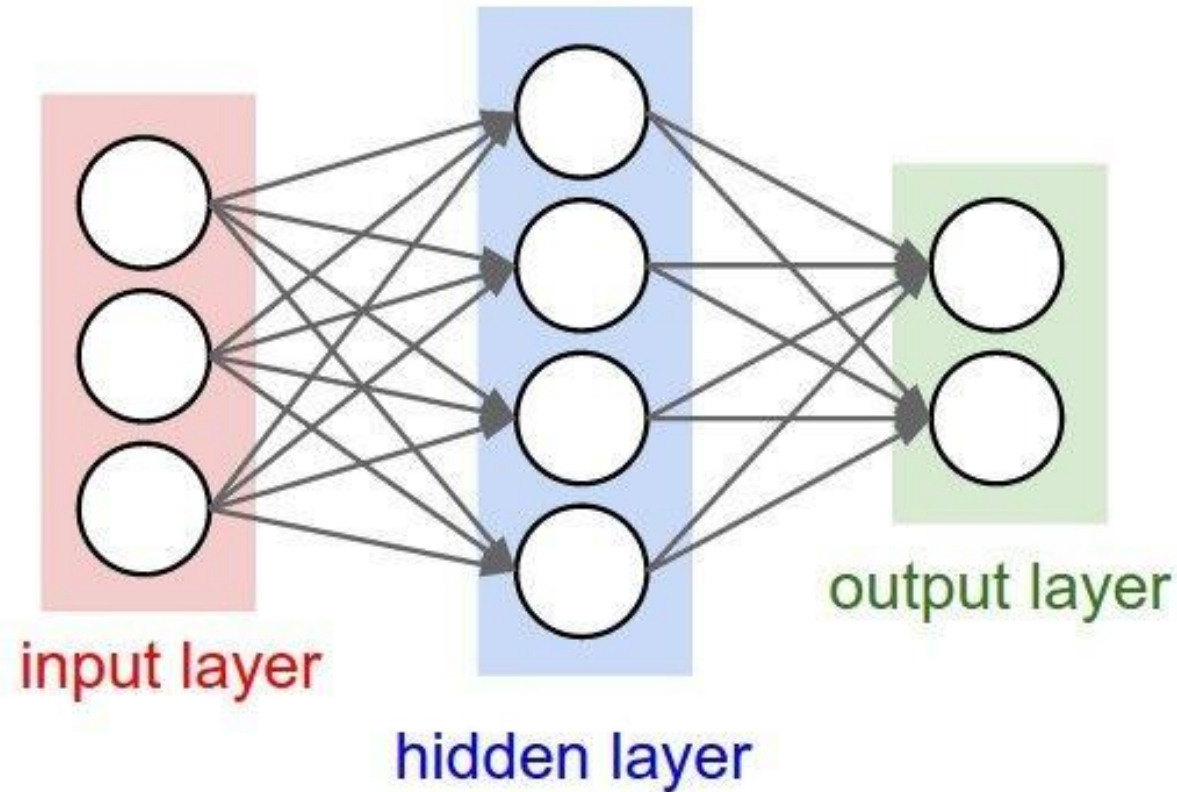
ReLU has No Gradient Vanishing Issue

ReLU: $f(x) = \max(0, x)$



Weight Initialization

- Q: what happens when $W=\text{constant}$ init is used?

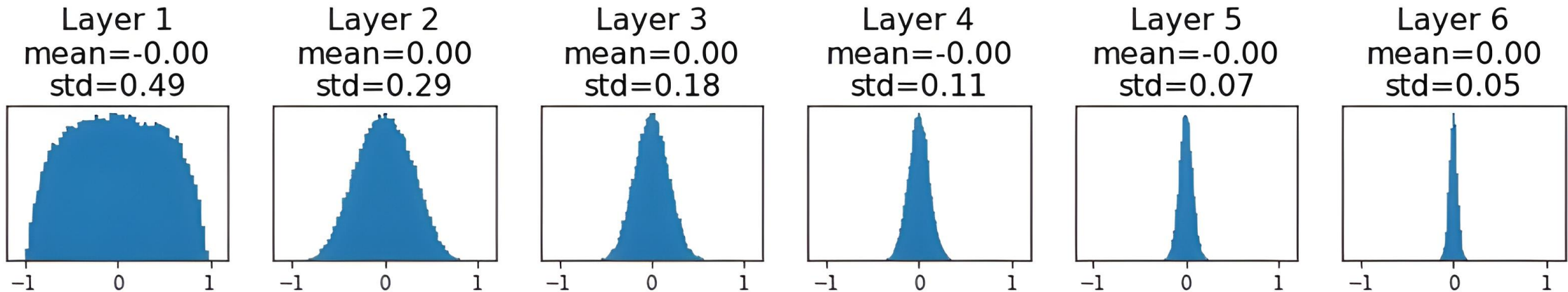


All neurons could be **updated in the same way**, which prevents them to learn different things.

Random Initialization with the Same Variance

```
dims = [4096] * 7      Forward pass for a 6-layer
hs = []                net with hidden size 4096
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = 0.01 * np.random.randn(Din, Dout)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

All activations tend to zero
for deep network layers



Random Initialization with the Same Variance

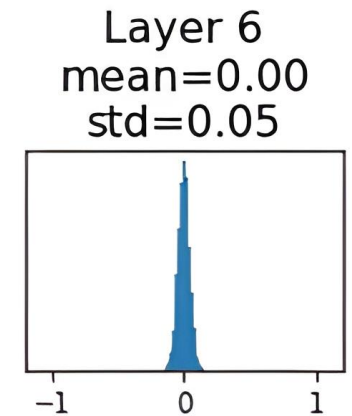
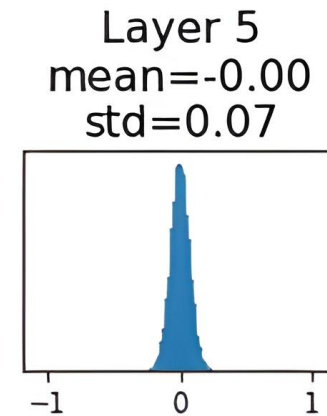
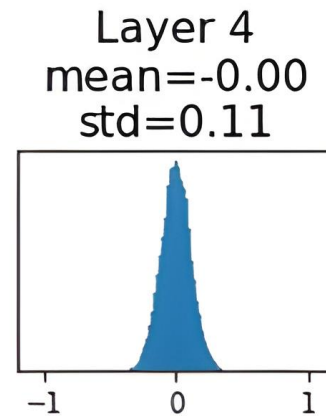
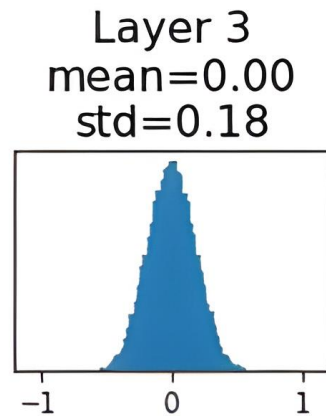
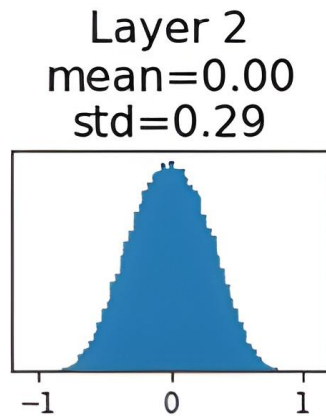
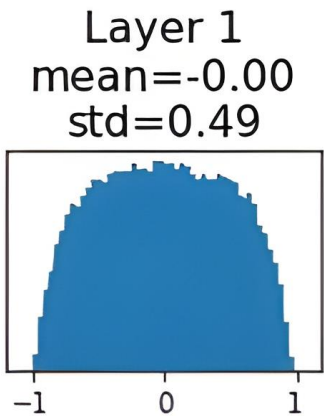
```
dims = [4096] * 7      Forward pass for a 6-layer
                        net with hidden size 4096
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = 0.01 * np.random.randn(Din, Dout)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

All activations tend to zero
for deep network layers

How about gradients?

$$x_{n+1} = \tanh(z_{n+1}) \quad z_{n+1} = w_n x_n + b$$

$$\frac{dx_{n+1}}{dw_n} = \frac{dx_{n+1}}{dz_{n+1}} \cdot x_n \approx 0$$



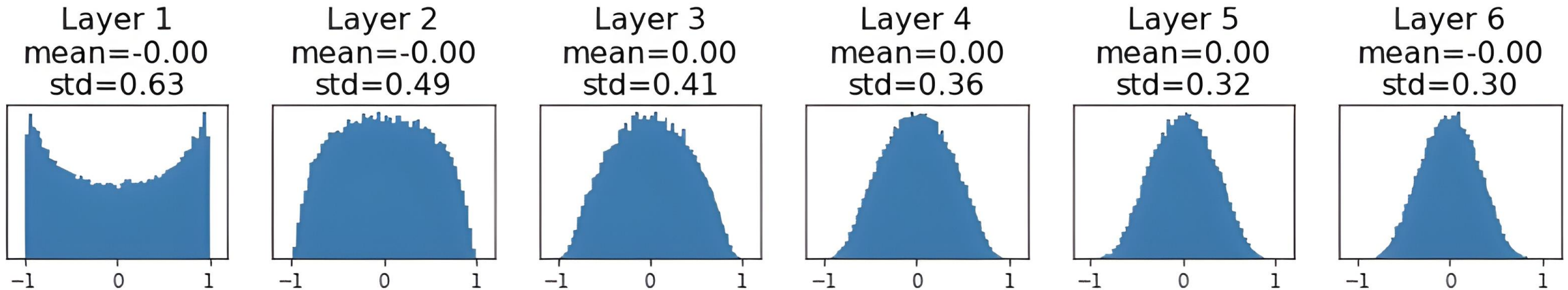
Xavier initialization

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

“Xavier” initialization:

$$std = \frac{1}{\sqrt{D_{in}}}$$

Each layer's output variance $\approx$ input variance



Kaiming Initialization

- Designed for ReLU activation

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) * np.sqrt(2/Din)
    x = np.maximum(0, x.dot(W))
    hs.append(x)
```

Kaiming initialization:

$$std = \sqrt{\frac{2}{D_{in}}}$$

